

AN INTRODUCTION TO GAUSSIAN PROCESS MODELS

by
Thomas Beckers
t.beckers@tum.de

Abstract

Within the past two decades, Gaussian process regression has been increasingly used for modeling dynamical systems due to some beneficial properties such as the bias variance trade-off and the strong connection to Bayesian mathematics. As data-driven method, a Gaussian process is a powerful tool for nonlinear function regression without the need of much prior knowledge. In contrast to most of the other techniques, Gaussian Process modeling provides not only a mean prediction but also a measure for the model fidelity. In this article, we give an introduction to Gaussian processes and its usage in regression tasks of dynamical systems. Try it yourself: gpr.tbeckers.com

Original Work: April, 2020
Current Revision: Feb 10, 2021

Chair of
Information-oriented Control
Technical University of Munich

Contents

1	Introduction	3
2	Gaussian Processes	3
2.1	Gaussian Process Regression	4
2.2	Multi-output Regression	6
2.3	Kernel-based View	7
2.4	Reproducing Kernel Hilbert Space	8
2.5	Model Error	12
2.5.1	Robust approach	13
2.5.2	Scenario approach	13
2.5.3	Information-theoretical approach	14
3	Model Selection	15
3.1	Kernel Functions	15
3.1.1	Constant Kernel	16
3.1.2	Linear Kernel	16
3.1.3	Polynomial Kernel	17
3.1.4	Matérn Kernel	17
3.1.5	Squared Exponential Kernel	17
3.1.6	Rational Quadratic Kernel	18
3.1.7	Squared Exponential ARD Kernel	18
3.2	Hyperparameter Optimization	20
3.2.1	Log Marginal Likelihood Approach	20
3.2.2	Cross-validation Approach	22
4	Gaussian Process Dynamical Models	22
4.1	Gaussian Process State Space Models	23
4.2	Gaussian Process Nonlinear Output Error Models	24
5	Summary	25
A	Conditional Distribution	26

1 Introduction

A Gaussian process (GP) is a stochastic process that is in general a collection of random variables indexed by time or space. Its special property is that any finite collection of these variables follows a multivariate Gaussian distribution. Thus, the GP is a distribution over infinitely many variables and, therefore, a distribution over functions with a continuous domain. Consequently, it describes a probability distribution over an infinite dimensional vector space. For engineering applications, the GP has gained increasing attention as supervised machine learning technique, where it is used as prior probability distribution over functions in Bayesian inference. The inference of continuous variables leads to Gaussian process regression (GPR) where the prior GP model is updated with training data to obtain a posterior GP distribution. Historically, GPR was used for the prediction of time series, at first presented by Wiener and Kolmogorov in the 1940's. Afterwards, it became increasingly popular in geostatistics in the 1970's, where GPR is known as *kriging*. Recently, it came back in the area of machine learning [Rad96; WR96], especially boosted by the rapidly increasing computational power.

In this article, we present background information about GPs and GPR, mainly based on [Ras06], focusing on the application in control. We start with an introduction of GPs, explain the role of the underlying kernel function and show its relation to reproducing kernel Hilbert spaces. Afterwards, the embedding in dynamical systems and the interpretation of the model uncertainty as error bounds is presented. Several examples are included for an intuitive understanding in addition to the formal notation.

2 Gaussian Processes

Let $(\Omega_{\text{ss}}, \mathcal{F}_{\sigma}, P)$ be a probability space with the sample space Ω_{ss} , the corresponding σ -algebra $\mathcal{F}_{\sigma}$ and the probability measure P . The index set is given by $\mathcal{Z} \subseteq \mathbb{R}^{n_z}$ with positive integer n_z . Then, a function $f_{\text{GP}}(\mathbf{z}, \omega_{\text{ss}})$, which is a measurable function of $\omega_{\text{ss}} \in \Omega_{\text{ss}}$ with index $\mathbf{z} \in \mathcal{Z}$, is called a stochastic process. The function $f_{\text{GP}}(\mathbf{z}, \omega_{\text{ss}})$ is a random variable on Ω_{ss} if $\mathbf{z} \in \mathcal{Z}$ is specified. It is simplified written as $f_{\text{GP}}(\mathbf{z})$. A GP is a stochastic process which is fully described by a mean function $m: \mathcal{Z} \rightarrow \mathbb{R}$ and covariance function $k: \mathcal{Z} \times \mathcal{Z} \rightarrow \mathbb{R}$ such that

$$f_{\text{GP}}(\mathbf{z}) \sim \mathcal{GP}(m(\mathbf{z}), k(\mathbf{z}, \mathbf{z}')) \quad (1)$$

$$\begin{aligned} m(\mathbf{z}) &= \mathbb{E}[f_{\text{GP}}(\mathbf{z})] \\ k(\mathbf{z}, \mathbf{z}') &= \mathbb{E}[(f_{\text{GP}}(\mathbf{z}) - m(\mathbf{z}))(f_{\text{GP}}(\mathbf{z}') - m(\mathbf{z}'))] \end{aligned} \quad (2)$$

with $\mathbf{z}, \mathbf{z}' \in \mathcal{Z}$. The covariance function is a measure for the correlation of two states $(\mathbf{z}, \mathbf{z}')$ and is called *kernel* in combination with GPs. Even though no analytic description of the probability density function of the GP exists in general, the interesting property is that any finite collection of its random variables $\{f_{\text{GP}}(\mathbf{z}_1), \dots, f_{\text{GP}}(\mathbf{z}_{n_{\text{GP}}})\}$ follows a n_{GP} -dimensional multivariate Gaussian distribution. As a GP defines a distribution over functions, each realization is also a function over the index set $\mathcal{Z}$.

Example 1. A GP $f_{\text{GP}}(t_c) \sim \mathcal{GP}(m(t_c), k(t_c, t'_c))$ with time $t_c \in \mathbb{R}_{\geq 0}$, where

$$m(t_c) = 1\text{A}, \quad k(t_c, t'_c) = \begin{cases} (0.1\text{A})^2 & t_c = t'_c \\ (0\text{A})^2 & t_c \neq t'_c \end{cases}$$

describes a time-dependent electric current signal with Gaussian white noise with a standard deviation of 0.1 A and a mean of 1 A.

2.1 Gaussian Process Regression

The GP can be utilized as prior probability distribution in Bayesian inference, which allows to perform function regression. Following the Bayesian methodology, new information is combined with existing information: using Bayes' theorem, the prior is combined with new data to obtain a posterior distribution. The new information is expressed as training data set $\mathcal{D} = \{X, Y\}$. It contains the input values $X = [\mathbf{x}_{\text{dat}}^{\{1\}}, \mathbf{x}_{\text{dat}}^{\{2\}}, \dots, \mathbf{x}_{\text{dat}}^{\{n_{\mathcal{D}}\}}] \in \mathcal{Z}^{1 \times n_{\mathcal{D}}}$ and output values $Y = [\tilde{y}_{\text{dat}}^{\{1\}}, \tilde{y}_{\text{dat}}^{\{2\}}, \dots, \tilde{y}_{\text{dat}}^{\{n_{\mathcal{D}}\}}]^\top \in \mathbb{R}^{n_{\mathcal{D}}}$, where

$$\tilde{y}_{\text{dat}}^{\{i\}} = f_{\text{GP}}(\mathbf{x}_{\text{dat}}^{\{i\}}) + \nu \quad (3)$$

for all $i = 1, \dots, n_{\mathcal{D}}$. The output data might be corrupted by Gaussian noise $\nu \sim \mathcal{N}(0, \sigma_n^2)$.

Remark 1. Note that we always use the standard notation X for the input training data and Y for the output training data throughout this report.

As any finite subset of a GP follows a multivariate Gaussian distribution, we can write the joint distribution

$$\begin{bmatrix} Y \\ f_{\text{GP}}(\mathbf{z}^*) \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(\mathbf{x}_{\text{dat}}^{\{1\}}) \\ \vdots \\ m(\mathbf{x}_{\text{dat}}^{\{n_{\mathcal{D}}\}}) \\ m(\mathbf{z}^*) \end{bmatrix}, \begin{bmatrix} K(X, X) + \sigma_n^2 I_{n_{\mathcal{D}}} & \mathbf{k}(\mathbf{z}^*, X) \\ \mathbf{k}(\mathbf{z}^*, X)^\top & k(\mathbf{z}^*, \mathbf{z}^*) \end{bmatrix} \right) \quad (4)$$

for any arbitrary test point $\mathbf{z}^* \in \mathcal{Z}$. The function $m: \mathcal{Z} \rightarrow \mathbb{R}$ denotes the mean function. The matrix function $K: \mathcal{Z}^{1 \times n_{\mathcal{D}}} \times \mathcal{Z}^{1 \times n_{\mathcal{D}}} \rightarrow \mathbb{R}^{n_{\mathcal{D}} \times n_{\mathcal{D}}}$ is called the covariance or *Gram matrix* with

$$K_{j,l}(X, X) = k(X_{:,l}, X_{:,j}) \text{ for all } j, l \in \{1, \dots, n_{\mathcal{D}}\} \quad (5)$$

where each element of the matrix represents the covariance between two elements of the training data X . The expression $X_{:,l}$ denotes the l -th column of X . For notational simplification, we shorten $K(X, X)$ to K when necessary. The vector-valued kernel function $\mathbf{k}: \mathcal{Z} \times \mathcal{Z}^{1 \times n_{\mathcal{D}}} \rightarrow \mathbb{R}^{n_{\mathcal{D}}}$ calculates the covariance between the test input $\mathbf{z}^*$ and the input training data X , i.e.,

$$\mathbf{k}(\mathbf{z}^*, X) = [k(\mathbf{z}^*, X_{:,1}), \dots, k(\mathbf{z}^*, X_{:,n_{\mathcal{D}}})]^\top. \quad (6)$$

To obtain the posterior predictive distribution of $f_{\text{GP}}(\mathbf{z}^*)$, we condition on the test point $\mathbf{z}^*$ and the training data set $\mathcal{D}$ given by

$$p(f_{\text{GP}}(\mathbf{z}^*)|\mathbf{z}^*, \mathcal{D}) = \frac{p(f_{\text{GP}}(\mathbf{z}^*), Y|X, \mathbf{z}^*)}{p(Y|X)}. \quad (7)$$

Thus, the conditional posterior Gaussian distribution is defined by the mean and the variance

$$\begin{aligned} \mu(f_{\text{GP}}(\mathbf{z}^*)|\mathbf{z}^*, \mathcal{D}) &= m(\mathbf{z}^*) + \mathbf{k}(\mathbf{z}^*, X)^\top (K + \sigma_n^2 I_{n_{\mathcal{D}}})^{-1} (Y - [m(X_{:,1}), \dots, m(X_{:,n_{\mathcal{D}}})]^\top) \\ \text{var}(f_{\text{GP}}(\mathbf{z}^*)|\mathbf{z}^*, \mathcal{D}) &= k(\mathbf{z}^*, \mathbf{z}^*) - \mathbf{k}(\mathbf{z}^*, X)^\top (K + \sigma_n^2 I_{n_{\mathcal{D}}})^{-1} \mathbf{k}(\mathbf{z}^*, X). \end{aligned} \quad (8)$$

A detailed derivation of the posterior mean and variance based on the joint distribution (4) can be found in appendix A. Analyzing (8) we can make the following observations:

i) The mean prediction can be written as

$$\mu(f_{\text{GP}}(\mathbf{z}^*)|\mathbf{z}^*, \mathcal{D}) = m(\mathbf{z}^*) + \sum_{j=1}^{n_{\mathcal{D}}} \alpha_j k(\mathbf{z}^*, X_{:,j}) \quad (9)$$

with $\boldsymbol{\alpha} = (K + \sigma_n^2 I_{n_{\mathcal{D}}})^{-1} (Y - [m(X_{:,1}), \dots, m(X_{:,n_{\mathcal{D}}})]^\top) \in \mathbb{R}^{n_{\mathcal{D}}}$. That formulation highlights the data-driven characteristic of the GPR as the posterior mean is a sum of kernel functions and its number grows with the number $n_{\mathcal{D}}$ of training data.

ii) The variance does not depend on the observed data, but only on the inputs, which is a property of the Gaussian distribution. The variance is the difference between two terms: The first term $k(\mathbf{z}^*, \mathbf{z}^*)$ is simply the prior covariance from which a (positive) term is subtracted, representing the information the observations contain about the function. The uncertainty of the prediction, expressed in the variance, holds only for $f_{\text{GP}}(\mathbf{z}^*)$ and does not consider the noise in the training data. For this purpose, an additional noise term $\sigma_n^2 I_{n_{\mathcal{D}}}$ can be added to the variance in (8). Finally, (8) clearly shows the strong dependence of the posterior mean and variance on the kernel k that we will discuss in depth in Section 3.

Example 2. We assume a GP with zero mean and a kernel function given by

$$k(z, z') = 0.3679^2 \exp\left(-\frac{(z - z')^2}{2 \cdot 2.7183^2}\right)$$

as prior distribution. The training data set $\mathcal{D}$ is assumed to be

$$X = \begin{bmatrix} 1 & 3 & 6 & 10 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & -0.3 & 0.3 & -0.2 \end{bmatrix}^\top,$$

where the output is corrupted by Gaussian noise with $\sigma_n = 0.0498$ standard deviation and the test point is assumed to be $z^* = 5$. According to (5) to (8) the Gram matrix $K(X, X)$ is calculated as

$$K(X, X) = \begin{bmatrix} 0.1378 & 0.1032 & 0.0249 & 0.0006 \\ 0.1032 & 0.1378 & 0.0736 & 0.0049 \\ 0.0249 & 0.0736 & 0.1378 & 0.0458 \\ 0.0006 & 0.0049 & 0.0458 & 0.1378 \end{bmatrix}$$

and the kernel vector $\mathbf{k}(z^*, X)$ and $k(z^*, z^*)$ are obtained to be

$$\begin{aligned}\mathbf{k}(z^*, X) &= [0.0458 \quad 0.1032 \quad 0.1265 \quad 0.0249] \\ k(z^*, z^*) &= 0.1378.\end{aligned}$$

Finally, with (8), we compute the predicted mean and variance for $f_{\text{GP}}(z^*)$

$$\mu(f_{\text{GP}}(z^*)|z^*, \mathcal{D}) = 0.0278, \quad \text{var}(f_{\text{GP}}(z^*)|z^*, \mathcal{D}) = 0.0015,$$

which is equivalent to a 2σ -standard deviation of 0.0775. Figure 1 shows the prior distribution (left), the posterior distribution with two training points (black crosses) in the middle, and the posterior distribution given the full training set $\mathcal{D}$ (right). The solid red line is the mean function and the gray shaded area indicates the 2σ -standard deviation. Five realizations (dashed lines) visualize the character of the distribution over functions.

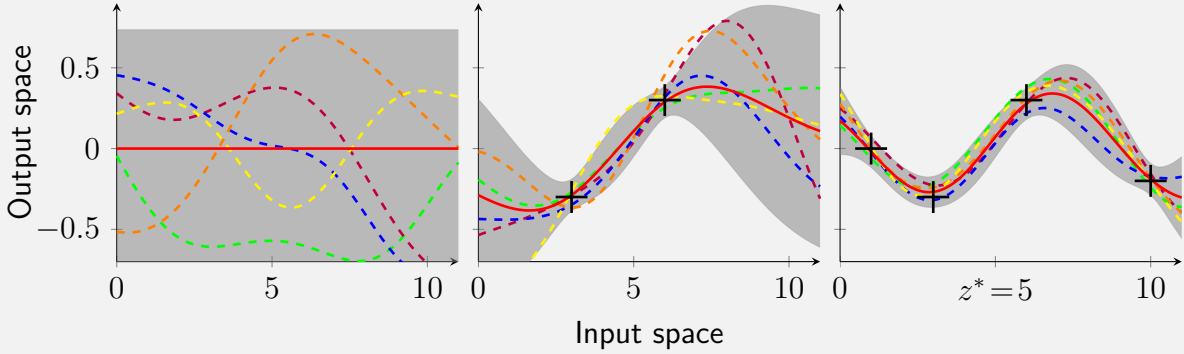


Figure 1: The prior distribution of a GP is updated with data that leads to the posterior distribution.

2.2 Multi-output Regression

So far, the GP regression allows functions with scalar outputs as in (8). For the extension to vector-valued outputs, multiple approaches exist: i) Extending the kernel to multivariate outputs [ÁRL12], ii) adding the output dimension as training data [Ber+17] or iii) using separated GPR for each output [Ras06]. While the first two approaches set a prior on the correlation between the output dimensions, the latter disregards a correlation without loss of generality. Following the approach in iii), the previous definition of the training set $\mathcal{D}$ is extended to a vector-valued output with

$$X = [\mathbf{x}_{\text{dat}}^{\{1\}}, \mathbf{x}_{\text{dat}}^{\{2\}}, \dots, \mathbf{x}_{\text{dat}}^{\{n_{\mathcal{D}}\}}] \in \mathcal{Z}^{1 \times n_{\mathcal{D}}}, \quad Y = [\tilde{\mathbf{y}}_{\text{dat}}^{\{1\}}, \tilde{\mathbf{y}}_{\text{dat}}^{\{2\}}, \dots, \tilde{\mathbf{y}}_{\text{dat}}^{\{n_{\mathcal{D}}\}}]^{\top} \in \mathbb{R}^{n_{\mathcal{D}} \times n_{y_{\text{dat}}}}, \quad (10)$$

where $n_{y_{\text{dat}}} \in \mathbb{N}$ is the dimension of the output and the vector-valued GP is defined by

$$\mathbf{f}_{\text{GP}}(\mathbf{z}) \sim \begin{cases} \mathcal{GP}(m^1(\mathbf{z}), k^1(\mathbf{z}, \mathbf{z}')) \\ \vdots \\ \mathcal{GP}(m^{n_{y_{\text{dat}}}}(\mathbf{z}), k^{n_{y_{\text{dat}}}}(\mathbf{z}, \mathbf{z}')) \end{cases} \quad (11)$$

$$\mathbf{m}(\mathbf{z}) := [m^1(\mathbf{z}), \dots, m^{n_{y_{\text{dat}}}}(\mathbf{z})]^{\top} \quad (12)$$

Following (4) to (8), we obtain for the predicted mean and variance

$$\begin{aligned}\mu(f_{\text{GP},i}(\mathbf{z}^*)|\mathbf{z}^*, \mathcal{D}) &= m^i(\mathbf{z}^*) + \mathbf{k}^i(\mathbf{z}^*, X)^\top (K^i + \sigma_{n,i}^2 I_{n_{\mathcal{D}}})^{-1} (Y_{:,i} - [m^i(X_{:,1}), \dots, m^i(X_{:,n_{\mathcal{D}}})]^\top) \\ \text{var}(f_{\text{GP},i}(\mathbf{z}^*)|\mathbf{z}^*, \mathcal{D}) &= k^i(\mathbf{z}^*, \mathbf{z}^*) - \mathbf{k}^i(\mathbf{z}^*, X)^\top (K^i + \sigma_{n,i}^2 I_{n_{\mathcal{D}}})^{-1} \mathbf{k}^i(\mathbf{z}^*, X)\end{aligned}\quad (13)$$

for each output dimension $i \in \{1, \dots, n_{\text{ydat}}\}$ with respect to the kernels $k^1, \dots, k^{n_{\text{ydat}}}$. The variable $\sigma_{n,i}$ denotes the standard deviation of the Gaussian noise that corrupts the i -th dimension of the output measurements. The n_{ydat} components of $\mathbf{f}_{\text{GP}}|\mathbf{z}^*, \mathcal{D}$ are combined into a multi-variable Gaussian distribution with

$$\begin{aligned}\boldsymbol{\mu}(\mathbf{f}_{\text{GP}}|\mathbf{z}^*, \mathcal{D}) &= [\mu(f_{\text{GP},1}|\mathbf{z}^*, \mathcal{D}), \dots, \mu(f_{\text{GP},n_{\text{ydat}}}|\mathbf{z}^*, \mathcal{D})]^\top \\ \Sigma(\mathbf{f}_{\text{GP}}|\mathbf{z}^*, \mathcal{D}) &= \text{diag}(\text{var}(f_{\text{GP},1}|\mathbf{z}^*, \mathcal{D}), \dots, \text{var}(f_{\text{GP},n_{\text{ydat}}}|\mathbf{z}^*, \mathcal{D})),\end{aligned}\quad (14)$$

where $\Sigma(\mathbf{f}_{\text{GP}}|\mathbf{z}^*, \mathcal{D})$ denotes the posterior variance matrix. This formulation allows to use a GP prior on vector-valued functions to perform predictions for test points $\mathbf{z}^*$. This approach treats each output dimension separately which is mostly sufficient and easy-to-handle. An alternative approach is to include the dimension as additional input, e.g., as in [Ber+17], with the benefit of a single GP at the price of loss of interpretability. For highly correlated output data, a multi-output kernel might be beneficial, see [ÁRL12].

Remark 2. *Without specific knowledge about a trend in the data, the prior mean functions $m^1, \dots, m^{n_{\text{ydat}}}$ are often set to zero, see [Ras06]. Therefore, we set the mean functions to zero for the remainder of the report if not stated otherwise.*

2.3 Kernel-based View

In Section 2.1, we target the GPR from a Bayesian perspective. However, for some applications of GPR a different point of view is beneficial; namely from the kernel perspective. In the following, we derive GPR from linear regression that is extended with a kernel transformation. In general, the prediction of parametric models is based on a parameter vector $\mathbf{w}$ which is typically learned using a set of training data points. In contrast, non-parametric models typically maintain at least a subset of the training data points in memory in order to make predictions for new data points. Many linear models can be transformed into a dual representation where the prediction is based on a linear combination of kernel functions. The idea is to transform the data points of a model to an often high-dimensional feature space where a linear regression can be applied to predict the model output, as depicted in Fig. 2. For a nonlinear feature map $\phi: \mathcal{Z} \rightarrow \mathcal{F}$, where $\mathcal{F}$ is a $n_\phi \in \mathbb{N} \cup \{\infty\}$ dimensional Hilbert space, the kernel function is given by the inner product $k(\mathbf{z}, \mathbf{z}') = \langle \phi(\mathbf{z}), \phi(\mathbf{z}') \rangle, \forall \mathbf{z}, \mathbf{z}' \in \mathcal{Z}$. Thus, the kernel implicitly encodes the way the data points are transformed into a higher dimensional space. The formulation as inner product in a feature space allows to extend many standard regression methods. Also the GPR can be derived using a standard linear regression model

$$f_{\text{lin}}(\mathbf{z}) = \mathbf{z}^\top \mathbf{w}, \quad \tilde{y}_{\text{dat}}^{\{i\}} = f_{\text{GP}}(\mathbf{x}_{\text{dat}}^{\{i\}}) + \nu \quad (15)$$

where $\mathbf{z} \in \mathcal{Z}$ is the input vector, $\mathbf{w} \in \mathbb{R}^{n_z}$ the vector of weights with $n_z = \dim(\mathcal{Z})$ and $f_{\text{lin}}: \mathcal{Z} \rightarrow \mathbb{R}$ the unknown function. The observed value $\tilde{y}_{\text{dat}}^{\{i\}} \in \mathbb{R}$ for the input $\mathbf{x}_{\text{dat}}^{\{i\}} \in \mathcal{Z}$

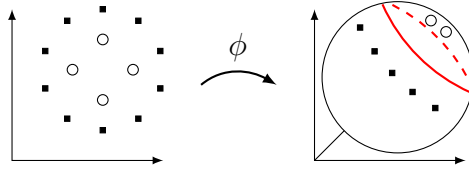


Figure 2: The mapping ϕ transforms the data points into a feature space where linear regressors can be applied to predict the output.

is corrupted by Gaussian noise $\nu \sim \mathcal{N}(0, \sigma_n^2)$ for all $i = 1, \dots, n_{\mathcal{D}}$. The analysis of this model is analogous to the standard linear regression, i.e., we put a prior on the weights such that $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \Sigma_p)$ with $\Sigma_p \in \mathbb{R}^{n_z \times n_z}$. Based on $n_{\mathcal{D}}$ collected training data points as defined in Section 2.1, that leads to the well known linear Bayesian regression

$$p(f_{\text{lin}}(\mathbf{z}^*) | \mathbf{z}^*, \mathcal{D}) = \mathcal{N}\left(\underbrace{\frac{1}{\sigma_n^2} \mathbf{z}^{*\top} A_{\text{lin}}^{-1} X Y}_{\mu(f_{\text{lin}}(\mathbf{z}^*) | \mathbf{z}^*, \mathcal{D})}, \underbrace{\mathbf{z}^{*\top} A_{\text{lin}}^{-1} \mathbf{z}^*}_{\text{var}(f_{\text{lin}}(\mathbf{z}^*) | \mathbf{z}^*, \mathcal{D})}\right) \quad (16)$$

where $A_{\text{lin}} = \sigma_n^{-2} X X^\top + \Sigma_p^{-1}$. Now, using the feature map $\phi(\mathbf{z})$ instead of $\mathbf{z}$ directly, leads to $f_{\text{GP}}(\mathbf{z}) = \phi(\mathbf{z})^\top \check{\mathbf{w}}$ with $\check{\mathbf{w}} \sim \mathcal{N}(\mathbf{0}, \check{\Sigma}_p)$, $\check{\Sigma}_p \in \mathbb{R}^{n_\phi \times n_\phi}$. As long as the projections are fixed functions, i.e., independent of the parameters w , the model is still linear in the parameters and, thus, analytically tractable. In particular, the Bayesian regression (16) with the mapping $\phi(\mathbf{z})$ can be written as

$$(f_{\text{GP}}(\mathbf{z}^*) | \mathbf{z}^*, \mathcal{D}) \sim \mathcal{N}\left(\frac{1}{\sigma_n^2} \phi(\mathbf{z}^*)^\top A_{\text{GP}}^{-1} [\phi(X_{:,1}); \dots; \phi(X_{:,n_{\mathcal{D}}})] Y, \phi(\mathbf{z}^*)^\top A_{\text{GP}}^{-1} \phi(\mathbf{z}^*)\right), \quad (17)$$

with the matrix $A_{\text{GP}} \in \mathbb{R}^{n_\phi \times n_\phi}$ given by

$$A_{\text{GP}} = \sigma_n^{-2} [\phi(X_{:,1}); \dots; \phi(X_{:,n_{\mathcal{D}}})] [\phi(X_{:,1}); \dots; \phi(X_{:,n_{\mathcal{D}}})]^\top + \check{\Sigma}_p^{-1}. \quad (18)$$

This equation can be simplified and rewritten to

$$(f_{\text{GP}}(\mathbf{z}^*) | \mathbf{z}^*, \mathcal{D}) \sim \mathcal{N}\left(\underbrace{\mathbf{k}(\mathbf{z}^*, X)^\top K^{-1} Y}_{\mu(f_{\text{GP}}(\mathbf{z}^*) | \mathbf{z}^*, \mathcal{D})}, \underbrace{\mathbf{k}(\mathbf{z}^*, \mathbf{z}^*) - \mathbf{k}(\mathbf{z}^*, X)^\top K^{-1} \mathbf{k}(\mathbf{z}^*, X)}_{\text{var}(f_{\text{GP}}(\mathbf{z}^*) | \mathbf{z}^*, \mathcal{D})}\right), \quad (19)$$

with $k(\mathbf{z}, \mathbf{z}') = \phi(\mathbf{z})^\top \check{\Sigma}_p \phi(\mathbf{z}')$ that equals (8). The fact that in (19) the feature map $\phi(\mathbf{z})$ is not needed is known as the *kernel trick*. This trick is also used in other kernel-based models, e.g., support vector machines (SVM), see [SC08] for more details.

2.4 Reproducing Kernel Hilbert Space

Even though a kernel neither uniquely defines the feature map nor the feature space, one can always construct a canonical feature space, namely the *reproducing kernel Hilbert space* (RKHS) given a certain kernel. After the introduction of the theory, illustrative examples for an intuitive understanding are presented. We will now formally present this construction procedure, starting with the concept of Hilbert spaces, following [BLG16]: A Hilbert space $\mathcal{F}$ represents all possible realizations of some class of functions, for example all functions of

continuity degree i , denoted by $\mathcal{C}^i$. Moreover, a Hilbert space is a vector space such that any function $f_{\mathcal{F}} \in \mathcal{F}$ must have a non-negative norm, $\|f_{\mathcal{F}}\|_{\mathcal{F}} > 0$ for $f_{\mathcal{F}} \neq 0$. All functions $f_{\mathcal{F}}$ must additionally be equipped with an inner-product in $\mathcal{F}$. Simply speaking, a Hilbert space is an infinite dimensional vector space, where many operations behave like in the finite case. The properties of Hilbert spaces have been explored in great detail in literature, e.g., in [DM+05]. An extremely useful property of Hilbert spaces is that they are equivalent to an associated kernel function [Aro50]. This equivalence allows to simply define a kernel, instead of fully defining the associated vector space. Formally speaking, if a Hilbert space $\mathcal{H}$ is a RKHS, it will have a unique positive definite kernel $k: \mathcal{Z} \times \mathcal{Z} \rightarrow \mathbb{R}$, which spans the space $\mathcal{H}$.

Theorem 1 (Moore-Aronszajn [Aro50]). *Every positive definite kernel k is associated with a unique RKHS $\mathcal{H}$.*

Theorem 2 ([Aro50]). *Let $\mathcal{F}$ be a Hilbert space, $\mathcal{Z}$ a non-empty set and $\phi: \mathcal{Z} \rightarrow \mathcal{F}$. Then, the inner product $\langle \phi(\mathbf{z}), \phi(\mathbf{z}') \rangle_{\mathcal{F}} := k(\mathbf{z}, \mathbf{z}')$ is positive definite.*

Importantly, any function $f_{\mathcal{H}}$ in $\mathcal{H}$ can be represented as a weighted linear sum of this kernel evaluated over the space $\mathcal{H}$, as

$$f_{\mathcal{H}}(\cdot) = \langle f_{\mathcal{H}}(\cdot), k(x, \cdot) \rangle_{\mathcal{H}} = \sum_{i=1}^{n_{\phi}} \alpha_i k(\mathbf{x}_{\text{dat}}^{\{i\}}, \cdot), \quad (20)$$

with $\alpha_i \in \mathbb{R}$ for all $i = \{1, \dots, n_{\phi}\}$, where $n_{\phi} \in \mathbb{N} \cup \{\infty\}$ is the dimension of the feature space $\mathcal{F}$. Thus, the RKHS is equipped with the inner-product

$$\langle f_{\mathcal{H}}, f'_{\mathcal{H}} \rangle_{\mathcal{H}} = \sum_{i=1}^{n_{\phi}} \sum_{j=1}^{n_{\phi}} \alpha_i \alpha'_j k(\mathbf{x}_{\text{dat}}^{\{i\}}, \mathbf{x}_{\text{dat}}^{\{j\}}), \quad (21)$$

with $f'_{\mathcal{H}}(\cdot) = \sum_{j=1}^{n_{\phi}} \alpha'_j k(\mathbf{x}_{\text{dat}}^{\{j\}}, \cdot) \in \mathcal{H}$, $\alpha'_j \in \mathbb{R}$. Now, the reproducing character manifests as

$$\forall \mathbf{z} \in \mathcal{Z}, \forall f_{\mathcal{H}} \in \mathcal{H}, \langle f_{\mathcal{H}}, k(x, \cdot) \rangle_{\mathcal{H}} = f_{\mathcal{H}}(\mathbf{z}), \text{ in particular } k(\mathbf{z}, \mathbf{z}') = \langle k(\cdot, \mathbf{z}), k(\cdot, \mathbf{z}') \rangle_{\mathcal{H}}. \quad (22)$$

According to [SHS06], the RKHS is then defined as

$$\mathcal{H} = \{f_{\mathcal{H}}: \mathcal{Z} \rightarrow \mathbb{R} | \exists \mathbf{c} \in \mathcal{F}, f_{\mathcal{H}}(\mathbf{z}) = \langle \mathbf{c}, \phi(\mathbf{z}) \rangle_{\mathcal{F}}, \forall \mathbf{z} \in \mathcal{Z}\}, \quad (23)$$

where $\phi(\mathbf{z})$ is the feature map constructing the kernel through $k(\mathbf{z}, \mathbf{z}') = \langle \phi(\mathbf{z}), \phi(\mathbf{z}') \rangle_{\mathcal{F}}$.

Example 3. We want to find the RKHS for the polynomial kernel with degree 2 that is given by

$$k(\mathbf{z}, \mathbf{z}') = (\mathbf{z}^{\top} \mathbf{z}')^2 = (z_1 z'_1)^2 + 2(z_1 z'_1 z_2 z'_2) + (z_2 z'_2)^2.$$

for any $\mathbf{z}, \mathbf{z}' \in \mathbb{R}^2$. First, we have to find a feature map ϕ such that the kernel corresponds to the inner product $k(\mathbf{z}, \mathbf{y}) = \langle \phi(\mathbf{z}), \phi(\mathbf{y}) \rangle$. A possible candidate for the feature

map is

$$\begin{aligned}\phi(\mathbf{z}) &= [z_1^2, \sqrt{2}z_1z_2, z_2^2]^\top, \text{ because} \\ \langle \phi(\mathbf{z}), \phi(\mathbf{z}') \rangle_{\mathbb{R}^3} &= \phi(\mathbf{z})^\top \phi(\mathbf{z}') = (z_1z_1')^2 + 2(z_1z_1'z_2z_2') + (z_2z_2')^2 = k(\mathbf{z}, \mathbf{z}').\end{aligned}$$

We know that the RKHS contains all linear combinations of the form

$$\begin{aligned}f_{\mathcal{H}}(\mathbf{z}) &= \sum_{i=1}^3 \alpha_i k(\mathbf{x}_{\text{dat}}^{\{i\}}, \mathbf{z}) = \sum_{i=1}^3 \alpha_i \langle \phi(\mathbf{z}'), \phi(\mathbf{z}) \rangle_{\mathbb{R}^3} = \sum_{i=1}^3 \langle \mathbf{c}, \phi(\mathbf{z}) \rangle_{\mathbb{R}^3} \\ &= c_1 z_1^2 + c_2 \sqrt{2} z_1 z_2 + c_3 z_2^2,\end{aligned}$$

with $\boldsymbol{\alpha}, \mathbf{c}, \mathbf{x}_{\text{dat}}^{\{i\}} \in \mathbb{R}^3$. Therefore, a possible candidate for the RKHS $\mathcal{H}$ is given by

$$\mathcal{H} = \left\{ f_{\mathcal{H}}: \mathbb{R}^2 \rightarrow \mathbb{R} \mid f_{\mathcal{H}}(\mathbf{z}) = c_1 z_1^2 + c_2 \sqrt{2} z_1 z_2 + c_3 z_2^2, \mathbf{c} \in \mathbb{R}^3 \right\} \quad (24)$$

Next, it must be checked if the proposed Hilbert space is the related RKHS to the polynomial kernel with degree 2. This is achieved in two steps: i) Checking if the space is a Hilbert space and ii) confirming the reproducing property. First, we can easily proof that this is a Hilbert space rewriting $f_{\mathcal{H}}(\mathbf{z}) = \mathbf{z}^\top S \mathbf{z}$ with symmetric matrix $S \in \mathbb{R}^{2 \times 2}$ and using the fact that $\mathcal{H}$ is euclidean and isomorphic to S . Second, the condition for an RKHS must be fulfilled, i.e., the reproducing property $f_{\mathcal{H}}(\mathbf{z}) = \langle f_{\mathcal{H}}(\cdot), k(\cdot, \mathbf{z}) \rangle_{\mathcal{H}}$. Since we can write

$$\langle f_{\mathcal{H}}(\cdot), k(\cdot, \mathbf{z}) \rangle_{\mathcal{H}} = \langle \mathbf{c}^\top \phi(\cdot), k(\cdot, \mathbf{z}) \rangle_{\mathcal{H}} = \sum_{i=1}^3 c_i k(\cdot, \mathbf{z}) = \mathbf{c}^\top \phi(\mathbf{z}) = f_{\mathcal{H}}(\mathbf{z}),$$

property (22) is fulfilled and, thus, $\mathcal{H}$ is the RKHS for the polynomial kernel with degree 2. Note that, even though the mapping ϕ is not unique for the kernel k , the relation of k and the RKHS $\mathcal{H}$ is unique.

Given a function $f_{\mathcal{H}} \in \mathcal{H}$ defined by $n_{\mathcal{D}}$ observations, its RKHS norm is defined as

$$\|f_{\mathcal{H}}\|_{\mathcal{H}}^2 = \langle f_{\mathcal{H}}, f_{\mathcal{H}} \rangle_{\mathcal{H}} = \sum_{i=1}^{n_{\mathcal{D}}} \sum_{j=1}^{n_{\mathcal{D}}} \alpha_i \alpha_j k(\mathbf{x}_{\text{dat}}^{\{i\}}, \mathbf{x}_{\text{dat}}^{\{j\}}) = \boldsymbol{\alpha}^\top K(X, X) \boldsymbol{\alpha}, \quad (25)$$

with $\boldsymbol{\alpha} \in \mathbb{R}^{n_{\mathcal{D}}}$ and $K(X, X)$ given by (5). We can also use the feature map such that

$$\|f_{\mathcal{H}}\|_{\mathcal{H}} = \inf \{ \|\mathbf{c}\|_{\mathcal{F}} : \mathbf{c} \in \mathcal{F}, f_{\mathcal{H}}(\mathbf{z}) = \langle \mathbf{c}, \phi(\mathbf{z}) \rangle_{\mathcal{F}}, \forall \mathbf{z} \in \mathcal{Z} \}. \quad (26)$$

As there is a unique relation between the RKHS $\mathcal{H}$ and the kernel k , the norm $\|f_{\mathcal{H}}\|_{\mathcal{H}}$ can equivalently be written as $\|f_{\mathcal{H}}\|_k$. The norm of a function in the RKHS indicates how fast the function varies over $\mathcal{Z}$ with respect to the geometry defined by the kernel. Formally, it can be written as

$$\frac{|f_{\mathcal{H}}(\mathbf{z}) - f_{\mathcal{H}}(\mathbf{z}')|}{d(\mathbf{z}, \mathbf{z}')} \leq \|f_{\mathcal{H}}\|_{\mathcal{H}}, \quad (27)$$

with the distance $d(\mathbf{z}, \mathbf{z}')^2 = k(\mathbf{z}, \mathbf{z}) - 2k(\mathbf{z}, \mathbf{z}') + k(\mathbf{z}', \mathbf{z}')$. A function with finite RKHS norm is also element of the RKHS. A more detailed discussion about RKHS and norms is given in [Wah90].

Example 4. We want to find the RKHS norm of a function $f_{\mathcal{H}}$ that is an element of the RKHS of the polynomial kernel with degree 2 that is given by

$$k(\mathbf{z}, \mathbf{z}') = (\mathbf{z}^\top \mathbf{z}')^2 = (z_1 z_1')^2 + 2(z_1 z_1' z_2 z_2') + (z_2 z_2')^2.$$

Let the function be

$$f_{\mathcal{H}}(\mathbf{z}) = \sum_{i=1}^3 \alpha_i k(\mathbf{x}_{\text{dat}}^{\{i\}}, \mathbf{z}), \text{ with} \quad (28)$$

$$\alpha_1 = 1, \alpha_2 = -2, \alpha_3 = 3 \quad (29)$$

$$\mathbf{x}_{\text{dat}}^{\{1\}} = [1, 1]^\top, \mathbf{x}_{\text{dat}}^{\{2\}} = [1, 2]^\top, \mathbf{x}_{\text{dat}}^{\{3\}} = [2, 1]^\top. \quad (30)$$

Hence, function (28) with (29) and (30) corresponds to

$$f_{\mathcal{H}}(\mathbf{z}) = 11z_1^2 + 6z_1z_2 - 4z_2^2.$$

Now, we have two possibilities how to calculate the RKHS norm. First, the RKHS-norm of $f_{\mathcal{H}}$ is calculated using (25) by

$$\|f_{\mathcal{H}}\|_{\mathcal{H}}^2 = \boldsymbol{\alpha}^\top K(X, X) \boldsymbol{\alpha} = \begin{bmatrix} 1 & -2 & 3 \end{bmatrix} \begin{bmatrix} 4 & 9 & 9 \\ 9 & 25 & 16 \\ 9 & 16 & 25 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 3 \end{bmatrix} = 155$$

with $X = [\mathbf{x}_{\text{dat}}^{\{1\}}, \mathbf{x}_{\text{dat}}^{\{2\}}, \mathbf{x}_{\text{dat}}^{\{3\}}]$. Alternatively, we can use (26) that results in $\|f_{\mathcal{H}}\|_{\mathcal{H}} = \|\mathbf{c}\|$, where $\mathbf{c}$ is defined by (24). Thus, the norm is computed as

$$f_{\mathcal{H}}(\mathbf{z}) = 11z_1^2 + 6z_1z_2 - 4z_2^2 \Rightarrow c_1 = 11, c_2 = \frac{6}{\sqrt{2}}, c_3 = -4 \Rightarrow \|f_{\mathcal{H}}\|_{\mathcal{H}}^2 = 155.$$

Example 5. In this example, we visualize the meaning of the RKHS norm. Figure 3 shows different quadratic functions with the same RKHS norm (top left and top right), a smaller RKHS norm (bottom left) and a larger RKHS norm (bottom right). An identical norm indicates a similar variation of the functions, whereas a higher norm leads to a more varying function.

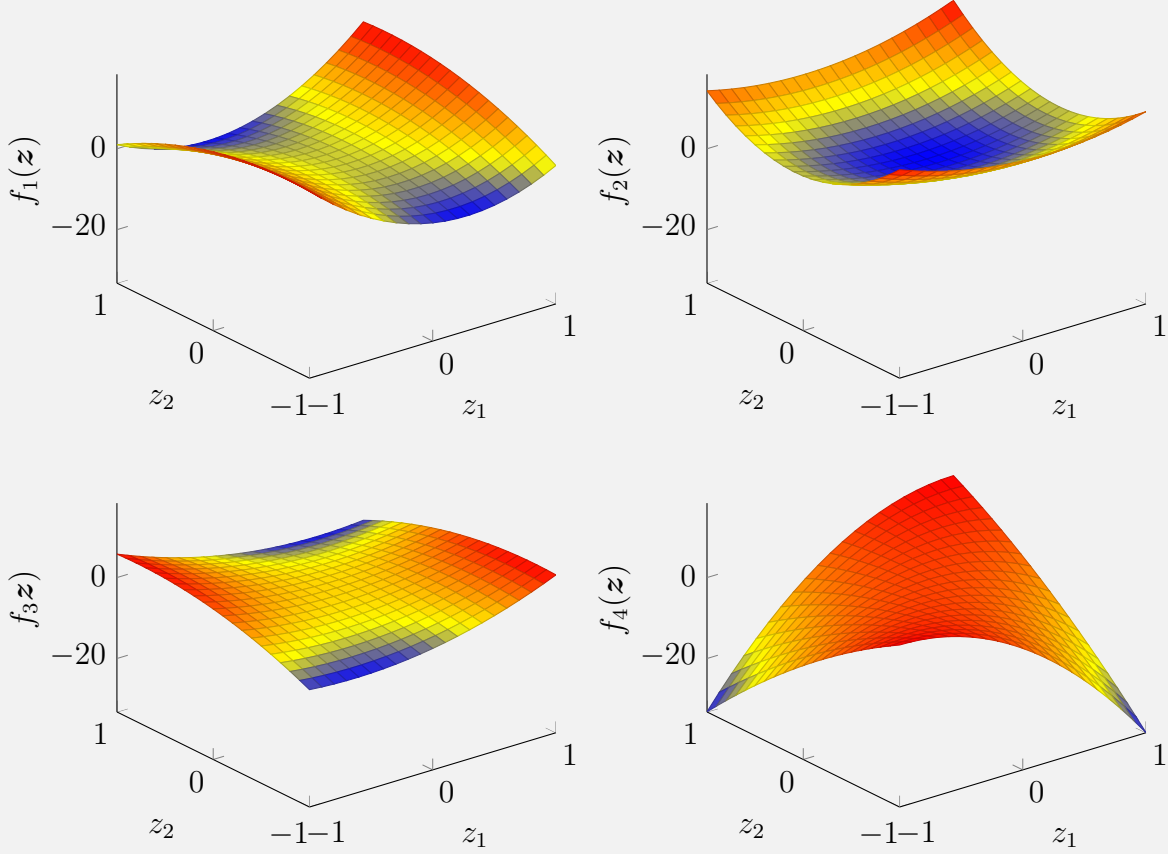


Figure 3: Functions with different RKHS-norms: $\|f_1\|_{\mathcal{H}}^2 = \|f_2\|_{\mathcal{H}}^2 = 4\|f_3\|_{\mathcal{H}}^2 = \frac{1}{2}\|f_4\|_{\mathcal{H}}^2$.

In summary, we investigate the unique relation between the kernel and its RKHS. The reproducing property allows us to write the inner-product as a tractable function which implicitly defines a higher (or even infinite) feature dimensional space. The RKHS-norm of a function is a Lipschitz-like indicator based on the metric defined by the kernel. This view of the RKHS is related to the kernel trick in machine learning. In the next section, the RKHS-norm is exploited to determine the error between the prediction of GPR and the actual data-generating function.

2.5 Model Error

One of the most interesting properties of GPR is the uncertainty description encoded in the predicted variance. This uncertainty is beneficial to quantify the error between the actual underlying data generating process and the GPR. In this section, we assume that there is an unknown function $f_{\text{uk}}: \mathbb{R}^{n_z} \rightarrow \mathbb{R}$ that generates the training data. In detail, the data

set $\mathcal{D} = \{X, Y\}$ consists of

$$\begin{aligned} X &= [\mathbf{x}_{\text{dat}}^{\{1\}}, \mathbf{x}_{\text{dat}}^{\{2\}}, \dots, \mathbf{x}_{\text{dat}}^{\{n_{\mathcal{D}}\}}] \in \mathbb{R}^{n_z \times n_{\mathcal{D}}} \\ Y &= [\tilde{y}_{\text{dat}}^{\{1\}}, \tilde{y}_{\text{dat}}^{\{2\}}, \dots, \tilde{y}_{\text{dat}}^{\{n_{\mathcal{D}}\}}]^\top \in \mathbb{R}^{n_{\mathcal{D}}}, \end{aligned} \quad (31)$$

where the data is generated by

$$\tilde{y}_{\text{dat}}^{\{i\}} = f_{\text{uk}}(\mathbf{x}_{\text{dat}}^{\{i\}}) + \nu, \nu \sim \mathcal{N}(0, \sigma_n^2) \quad (32)$$

for all $i = \{1, \dots, n_{\mathcal{D}}\}$. Without any assumptions on f_{uk} it is obviously not possible to quantify the model error. Loosely speaking, the prior distribution of the GPR with kernel k must be suitable to learn the unknown function. More technically, f_{uk} must be an element of the RKHS spanned by the kernel as described in (23). This leads to the following assumption.

Assumption 1. *The function f_{uk} has a finite RKHS norm with respect to the kernel k , i.e., $\|f_{\text{uk}}\|_{\mathcal{H}} < \infty$, where $\mathcal{H}$ is the RKHS spanned by k .*

This sounds paradoxical as f_{uk} is assumed to be unknown. However, there exist kernels that can approximate any continuous function arbitrarily exact. Thus, for any continuous function, an arbitrarily close function is element of the RKHS of an universal kernel. For more details, we refer to Section 2.4. More information about the model error of misspecified Gaussian Process model can be found in [BUH18]

We classify the error quantification in three different approaches: i) the robust approach, ii) the scenario approach, and iii) the information-theoretical approach. The different techniques are presented in the following and visualized in Fig. 4. For the remainder of this section, we assume that a GPR is trained with the data set (31) and Assumption 1 holds.

2.5.1 Robust approach

The robust approach exploits the fact that the prediction of the GPR is Gaussian distributed. Thus, for any $\mathbf{z}^* \in \mathbb{R}^{n_z}$, the model error is bounded by

$$|f_{\text{uk}}(\mathbf{z}^*) - \mu(f_{\text{GP}}|\mathbf{z}^*, \mathcal{D})| \leq c \text{var}(f_{\text{GP}}|\mathbf{z}^*, \mathcal{D}) \quad (33)$$

with high probability where $c \in \mathbb{R}_{>0}$ adjusts the probability. However, for multiple test points $\mathbf{z}_1^*, \mathbf{z}_2^*, \dots \in \mathbb{R}^{n_z}$, this approach neglects any correlation between $f_{\text{GP}}(\mathbf{z}_1^*), f_{\text{GP}}(\mathbf{z}_2^*), \dots$. Figure 4 shows how for a given $\mathbf{z}_1^*$ and $\mathbf{z}_2^*$, the variance is exploited as upper bound. Thus, any prediction is handled independently, which leads to a very conservative bound, see [UBH18].

2.5.2 Scenario approach

Instead of using the mean and the variance as in the robust approach, the scenario approach deals with the samples of the GPR directly. In contrast to the other methods, there is no direct model error quantification but rather a sample based quantification. The idea is to draw a large number $n_{\text{scen}} \in \mathbb{N}$ of sample functions $f_{\text{GP}}^1, f_{\text{GP}}^2, \dots, f_{\text{GP}}^{n_{\text{scen}}}$ over $n_s \in \mathbb{N}$ sampling points. The sampling is performed by drawing multiple instances from f_{GP} given by the

multivariate Gaussian distribution

$$\begin{bmatrix} Y \\ f_{\text{GP}}(\mathbf{z}_1^*) \\ \vdots \\ f_{\text{GP}}(\mathbf{z}_{n_s}^*) \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} m(\mathbf{x}_{\text{dat}}^{\{1\}}) \\ \vdots \\ m(\mathbf{x}_{\text{dat}}^{\{n_{\mathcal{D}}\}}) \\ m(\mathbf{z}_1^*) \\ \vdots \\ m(\mathbf{z}_{n_s}^*) \end{bmatrix}, \begin{bmatrix} K(X, X) + \sigma_n^2 I_{n_{\mathcal{D}}} & K(X^*, X) \\ K(X^*, X)^\top & K(X^*, X^*) \end{bmatrix} \right), \quad (34)$$

where $X^* = [\mathbf{z}_1^*, \dots, \mathbf{z}_{n_s}^*]$ contains the sampling points. Each sample can then be used in the application instead of the unknown function. For a large number of samples it is assumed that the unknown function is close to one of these samples. However, the crux of this approach is to determine, for a given model error $c \in \mathbb{R}_{>0}$, the required number of samples n_{scen} and probability $\delta_{\text{scen}} > 0$ such that

$$\mathbb{P} \left(|f_{\text{uk}}(\mathbf{z}^*) - f_{\text{GP}}^i(\mathbf{z}^*)| \leq c, i \in \{1, \dots, n_{\text{scen}}\} \right) \geq \delta_{\text{scen}} \quad (35)$$

for all $\mathbf{z}^* \in Z$. In Fig. 4, five different samples of a GP model are drawn as example.

2.5.3 Information-theoretical approach

Alternatively, the work in [Sri+12] derives an upper bound for samples of the GPR on a compact set with a specific probability. In contrast to the robust approach, the correlation between the function values are considered. We restate here the theorem of [Sri+12].

Theorem 3 ([Sri+12]). *Given Assumption 1, the model error $\Delta \in \mathbb{R}$*

$$\Delta = |\mu(f_{\text{GP}}|\mathbf{z}, \mathcal{D}) - f_{\text{uk}}(\mathbf{z})| \quad (36)$$

is bounded for all $\mathbf{z}$ on a compact set $\Omega \subset \mathbb{R}^{n_z}$ with a probability of at least $\delta \in (0, 1)$ by

$$P \left\{ \forall \mathbf{z} \in \Omega, \Delta \leq |\beta \Sigma^{\frac{1}{2}}(f_{\text{GP}}|\mathbf{z}, \mathcal{D})| \right\} \geq \delta, \quad (37)$$

where $\beta \in \mathbb{R}$ is defined as

$$\beta = \sqrt{2\|f_{\text{uk}}\|_k^2 + 300\gamma_{\text{max}} \ln^3 \left(\frac{n_{\mathcal{D}} + 1}{1 - \delta} \right)}. \quad (38)$$

The variable $\gamma_{\text{max}} \in \mathbb{R}$ is the maximum of the information gain

$$\gamma_{\text{max}} = \max_{\mathbf{x}_{\text{dat}}^{\{1\}}, \dots, \mathbf{x}_{\text{dat}}^{\{n_{\mathcal{D}}+1\}} \in \Omega} \frac{1}{2} \log |I_{n_{\mathcal{D}}+1} + \sigma_n^{-2} K(\mathbf{z}, \mathbf{z}')| \quad (39)$$

with Gram matrix $K(\mathbf{z}, \mathbf{z}')$ and the input elements $\mathbf{z}, \mathbf{z}' \in \{\mathbf{x}_{\text{dat}}^{\{1\}}, \dots, \mathbf{x}_{\text{dat}}^{\{n_{\mathcal{D}}+1\}}\}$.

To compute this bound, the RKHS norm of f_{uk} must be known. That is in application usually not the case. However, often the norm can be upper bounded and thus, the bound

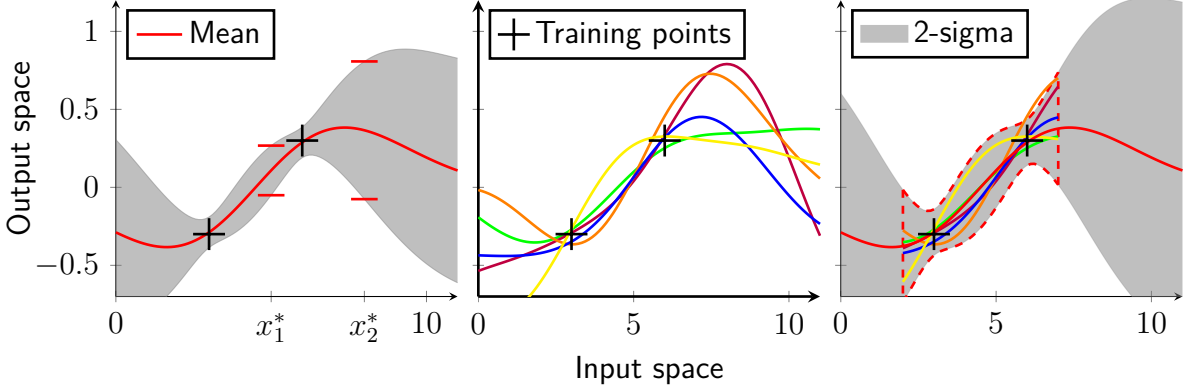


Figure 4: Different approaches to quantify the model error: Robust approach (left), scenario approach (middle), information-theoretical approach (right).

in Theorem 3 can be upper bounded. For this purpose, the relation of the RKHS norm to the Lipschitz constant given by (27) is beneficial as the Lipschitz constant is more likely to be known. In general, the computation of the information gain is a non-convex optimization problem. However, the information capacity $\gamma_{\max}$ has a sub-linear dependency on the number of training points for many commonly used kernel functions [Sri+12]. Therefore, even though β is increasing with the number of training data, it is possible to learn the true function f_{uk} arbitrarily exactly [Ber+16]. In contrast to the other approaches, Theorem 3 allows to bound the error for any test point in a compact set. In [BKH19], we exploit this approach in GP model based control tasks. The right illustration of Fig. 4 visualizes the information-theoretical bound.

3 Model Selection

Equation (8) clearly shows the immense impact of the kernel on the posterior mean and variance. However, this is not surprising as the kernel is an essential part of the prior model. For practical applications that leads to the question how to choose the kernel. Additionally, most kernels depend on a set of hyperparameters that must be defined. Thus in order to turn GPR into a powerful practical tool it is essential to develop methods that address the model selection problem. We see the model selection as the determination of the kernel and its hyperparameters. We only focus on kernels that are defined on $\mathcal{Z} \subseteq \mathbb{R}^{n_z}$. In the next two subsections, we present different kernels and explain the role of the hyperparameters and their selection, mainly based on [Ras06].

Remark 3. *The selection of the kernel functions seems to be similar to the model selection for parametric models. However, there are two major differences: i) the selection is fully covered by the Bayesian methodology and ii) many kernels allow to model a wide range of different functions whereas parametric models are typically limited to very specific types of functions.*

3.1 Kernel Functions

The value of the kernel function $k(\mathbf{z}, \mathbf{z}')$ is an indicator of the interaction of two states $(\mathbf{z}, \mathbf{z}')$. Thus, an essential part of GPR is the selection of the kernel function and the estimation of its

free parameters $\varphi_1, \varphi_2, \dots, \varphi_{n_\varphi}$, called hyperparameters. The number n_φ of hyperparameters depends on the kernel function. The choice of the kernel function and the determination of the corresponding hyperparameters can be seen as degrees of freedom of the regression. First of all, we start with the general properties of a function to be qualified as a kernel for GPR. A necessary and sufficient condition for the function $k: \mathcal{Z} \times \mathcal{Z} \rightarrow \mathbb{R}$ to be a valid kernel is that the Gram matrix, see (5), is positive semidefinite for all possible input values [SC04].

Remark 4. *As shown in Section 2.4, the kernel function must be positive definite to span an unique RKHS. That seems to be contradictory to the required positive semi-definiteness of the Gram matrix. The solution is the definition of positive definite kernels as it is equivalent to a positive semi-definite Gram matrix. In detail, a symmetric function $k: \mathcal{Z} \times \mathcal{Z} \rightarrow \mathbb{R}$ is a positive definite kernel on $\mathcal{Z}$ if*

$$\sum_{j=1}^{n_{\mathcal{D}}} \sum_{i=1}^{n_{\mathcal{D}}} k(\mathbf{x}_{dat}^{\{i\}}, \mathbf{x}_{dat}^{\{j\}}) c_i c_j \geq 0 \quad (40)$$

holds for any $n_{\mathcal{D}} \in \mathbb{N}$, $\mathbf{x}_{dat}^{\{1\}}, \dots, \mathbf{x}_{dat}^{\{n_{\mathcal{D}}\}} \in \mathcal{Z}$ and $c_1, \dots, c_n \in \mathbb{R}$. Thus, there exists a positive semi-definite matrix $A_G \in \mathbb{R}^{n_{\mathcal{D}} \times n_{\mathcal{D}}}$ such that

$$\mathbf{x}_{dat}^\top A_G \mathbf{x}_{dat} = \sum_{j=1}^{n_{\mathcal{D}}} \sum_{i=1}^{n_{\mathcal{D}}} k(\mathbf{x}_{dat}^{\{i\}}, \mathbf{x}_{dat}^{\{j\}}) c_i c_j \quad (41)$$

holds for any $n_{\mathcal{D}} \in \mathbb{N}$ and $\mathbf{z} \in \mathcal{Z}$.

The set of functions k which fulfill this condition is denoted with $\mathcal{K}$. Kernel functions can be separated into two classes, the *stationary* and *non-stationary* kernels. A stationary kernel is a function of the distance $\mathbf{z} - \mathbf{z}'$. Thus, it is invariant to translations in the input space. In contrast, non-stationary kernels depend directly on $\mathbf{z}, \mathbf{z}'$ and are often functions of a dot product $\mathbf{z}^\top \mathbf{z}$. In the following, we list some common kernel functions with their basic properties. Even though the number of presented kernels is limited, new kernels can be constructed easily as $\mathcal{K}$ is closed under specific operations such as addition and scalar multiplication. At the end, we summarize the equation of each kernel in Table 1 and provide a comparative example.

3.1.1 Constant Kernel

The equation for the constant kernel is given by

$$k(\mathbf{z}, \mathbf{z}') = \varphi_1^2. \quad (42)$$

This kernel is mostly used in addition to other kernel functions. It depends on a single hyperparameter $\varphi_1 \in \mathbb{R}_{\geq 0}$.

3.1.2 Linear Kernel

The equation for the linear kernel is given by

$$k(\mathbf{z}, \mathbf{z}') = \mathbf{z}^\top \mathbf{z}'. \quad (43)$$

The linear kernel is a dot-product kernel and thus, non-stationary. The kernel can be obtained from Bayesian linear regression as shown in Section 2.3. The linear kernel is often used in combination with the constant kernel to include a bias.

3.1.3 Polynomial Kernel

The equation for the polynomial kernel is given by

$$k(\mathbf{z}, \mathbf{z}') = (\mathbf{z}^\top \mathbf{z}' + \varphi_1^2)^p, \quad p \in \mathbb{N}. \quad (44)$$

The polynomial kernel has an additional parameter $p \in \mathbb{N}$, that determines the degree of the polynomial. Since a dot product is contained, the kernel is also non-stationary. The prior variance grows rapidly for $\|\mathbf{z}\| > 1$ such that the usage for some regression problems is limited. It depends on a single hyperparameter $\varphi_1 \in \mathbb{R}_{\geq 0}$.

3.1.4 Matérn Kernel

The equation for the Matérn kernel is given by

$$k(\mathbf{z}, \mathbf{z}') = \varphi_1^2 \exp\left(-\frac{\sqrt{2\check{p}}\|\mathbf{z} - \mathbf{z}'\|}{\varphi_2}\right) \frac{p!}{(2p)!} \sum_{i=0}^p \frac{(p+i)!}{i!(p-i)!} \left(\frac{\sqrt{8\check{p}}\|\mathbf{z} - \mathbf{z}'\|}{\varphi_2}\right)^{p-i} \quad (45)$$

with $\check{p} = p + \frac{1}{2}$, $p \in \mathbb{N}$. The Matérn kernel is a very powerful kernel and presented here with the most common parameterization for $\check{p}$. Functions drawn from a GP model with Matérn kernel are p -times differentiable. The more general equation of this stationary kernel can be found in [Bis06]. This kernel is an *universal kernel* which is explained in the following.

Lemma 1 ([SC08, Lemma 4.55]). *Consider the RKHS $\mathcal{H}(\mathcal{Z}_c)$ of an universal kernel on any prescribed compact subset $\mathcal{Z}_c \in \mathcal{Z}$. Given any positive number ε and any function $f_c \in \mathcal{C}^1(\mathcal{Z}_c)$, there is a function $f_{\mathcal{H}} \in \mathcal{H}(\mathcal{Z}_c)$ such that $\|f_c - f_{\mathcal{H}}\|_{\mathcal{Z}_c} \leq \varepsilon$.*

Intuitively speaking, a GPR with an universal kernel can approximate any continuous function arbitrarily exact on a compact set. For $p \rightarrow \infty$, it results in the squared exponential kernel. The two hyperparameters are $\varphi_1 \in \mathbb{R}_{\geq 0}$ and $\varphi_2 \in \mathbb{R}_{> 0}$.

3.1.5 Squared Exponential Kernel

The equation for the squared exponential kernel is given by

$$k(\mathbf{z}, \mathbf{z}') = \varphi_1^2 \exp\left(-\frac{\|\mathbf{z} - \mathbf{z}'\|^2}{2\varphi_2^2}\right). \quad (46)$$

Probably the most widely used kernel function for GPR is the squared exponential kernel, see [Ras06]. The hyperparameter φ_1 describes the signal variance which determines the average distance of the data-generating function from its mean. The lengthscale φ_2 defines how far it is needed to move along a particular axis in input space for the function values to become uncorrelated. Formally, the lengthscale determines the number of expected upcrossings of the level zero in a unit interval by a zero-mean GP. The squared exponential kernel is infinitely differentiable, which means that the GPR exhibits a smooth behavior. As limit of the Matérn kernel, it is also an universal kernel, see [MXZ06].

Example 6. Figure 5 shows the power for regression of universal kernel functions. In this example, a GPR with squared exponential kernel is used for different training data sets. The hyperparameter are optimized individually for each training data set by means of the likelihood, see Section 3.2. Note that all presented regressions are based on the **same** GP model, i.e. the same kernel function, but with different data sets. That highlights again the superior flexibility of GPR.

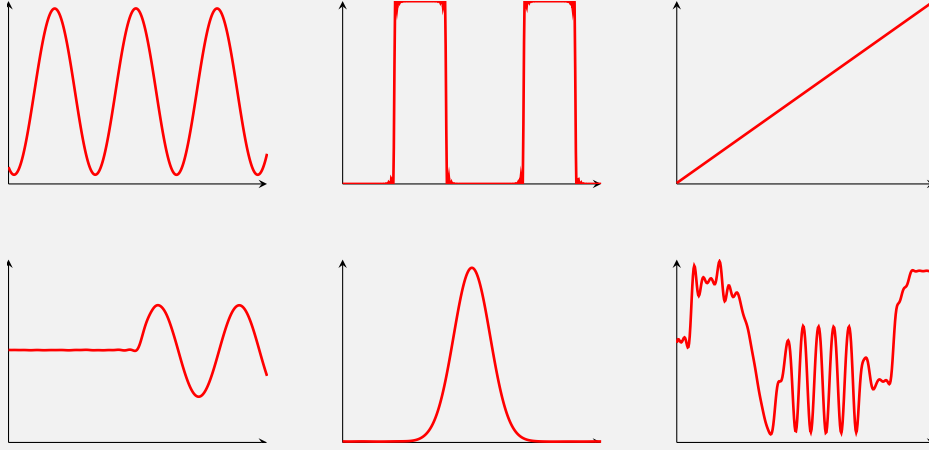


Figure 5: Examples for the flexibility of the regression that all are based on the same GP model.

3.1.6 Rational Quadratic Kernel

The equation for the rational quadratic kernel is given by

$$k(\mathbf{z}, \mathbf{z}') = \varphi_1^2 \left(1 + \frac{\|\mathbf{z} - \mathbf{z}'\|^2}{2p\varphi_2^2} \right)^{-p}, \quad p \in \mathbb{N}. \quad (47)$$

This kernel is equivalent to summing over infinitely many squared exponential kernels with different lengthscales. Hence, GP priors with this kernel are expected to see functions which vary smoothly across many lengthscales. The parameter p determines the relative weighting of large-scale and small-scale variations. For $p \rightarrow \infty$, the rational quadratic kernel is identical to the squared exponential kernel.

3.1.7 Squared Exponential ARD Kernel

The equation for the squared exponential ARD kernel is given by

$$k(\mathbf{z}, \mathbf{z}') = \varphi_1^2 \exp \left(-(\mathbf{z} - \mathbf{z}')^\top P^{-1} (\mathbf{z} - \mathbf{z}') \right), \quad P = \text{diag}(\varphi_2^2, \dots, \varphi_{1+n_z}^2). \quad (48)$$

The *automatic relevance determination* (ARD) extension to the squared exponential kernel allows for independent lengthscales $\varphi_2, \dots, \varphi_{1+n_z} \in \mathbb{R}_{>0}$ for each dimension of $\mathbf{z}, \mathbf{z}' \in \mathbb{R}^{n_z}$. The individual lengthscales are typically larger for dimensions which are irrelevant as the covariance will become almost independent of that input. A more detailed discussion about the advantages of different kernels can be found, for instance, in [Mac97] and [Bis06].

Example 7. In this example, we use three GPRs with the same set of training data

$$X = [1, 3, 5, 7, 9], Y = [0, 1, 2, 3, 6] \quad (49)$$

but with different kernels, namely the squared exponential (46), the linear (43), and the polynomial (44) kernel. Figure 6 shows the different shapes of the regressions with the posterior mean (red), the posterior variance (gray shaded) and the training points (black). Even for this simple data set, the flexibility of the squared exponential kernel is already visible.

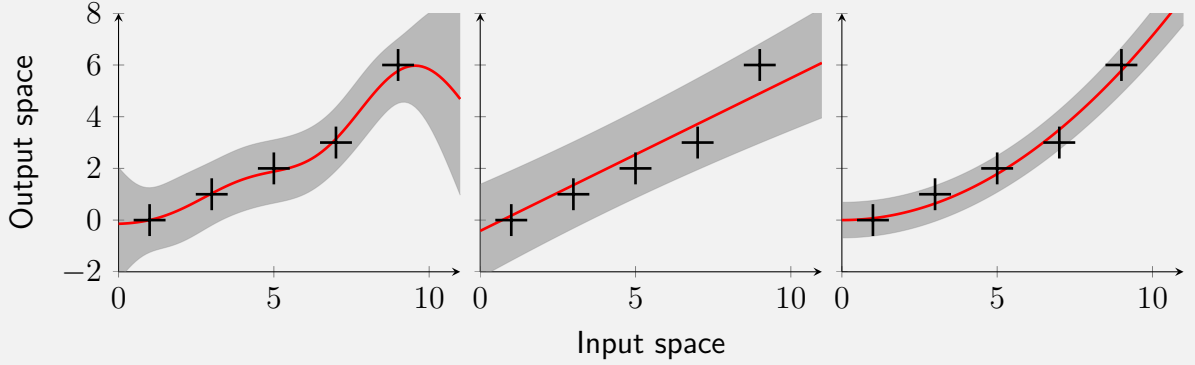


Figure 6: GPR with different kernels: squared exponential (left), linear (middle) and polynomial with degree 2 (right).

Kernel name	$k(\mathbf{z}, \mathbf{z}') =$
Constant	φ_1^2
Linear	$\mathbf{z}^\top \mathbf{z}' + \varphi_1^2$
Polynomial $p \in \mathbb{N}$	$(\mathbf{z}^\top \mathbf{z}' + \varphi_1^2)^p$
Matérn $\check{p} = p + \frac{1}{2}, p \in \mathbb{N}$	$\varphi_1^2 \exp\left(-\frac{\sqrt{2\check{p}}\ \mathbf{z}-\mathbf{z}'\ }{\varphi_2}\right) \frac{p!}{(2p)!} \sum_{i=0}^p \frac{(p+i)!}{i!(p-i)!} \left(\frac{\sqrt{8\check{p}}\ \mathbf{z}-\mathbf{z}'\ }{\varphi_2}\right)^{p-i}$
Squared exponential	$\varphi_1^2 \exp\left(-\frac{\ \mathbf{z}-\mathbf{z}'\ ^2}{2\varphi_2^2}\right)$
Rational quadratic	$\varphi_1^2 \left(1 + \frac{\ \mathbf{z}-\mathbf{z}'\ ^2}{2p\varphi_2^2}\right)^{-p}$
Squared exponential ARD	$\varphi_1^2 \exp\left(-(\mathbf{z}-\mathbf{z}')^\top P^{-1}(\mathbf{z}-\mathbf{z}')\right), P = \text{diag}(\varphi_2^2, \dots, \varphi_{1+n_z}^2)$

Table 1: Overview of some commonly used kernel functions.

3.2 Hyperparameter Optimization

In addition to the selection of a kernel function, values for any hyperparameter must be determined to perform the regression. The number of hyperparameters depends on the kernel function used. We concatenate all hyperparameters in a vector $\boldsymbol{\varphi}$ with size $n_\varphi \in \mathbb{N}$, where $\boldsymbol{\varphi} \in \Phi \subseteq \mathbb{R}^{n_\varphi}$. The hyperparameter set Φ is introduced to cover the different spaces of the individual hyperparameters as defined in the following.

Definition 1. The set Φ is called a hyperparameter set for a kernel function k if and only if the set Φ is a domain for the hyperparameters $\boldsymbol{\varphi}$ of k .

Often, the signal noise σ_n^2 , see (4), is also treated as hyperparameter. For a better understanding, we keep the signal noise separated from the hyperparameters. There exist several techniques that allow computing the hyperparameters and the signal noise with respect to one optimality criterion. From a Bayesian perspective, we want to find the vector of hyperparameters $\boldsymbol{\varphi}$ which are most likely for the output data Y given the inputs X and a GP model. For this purpose, one approach is to optimize the *log marginal likelihood function* of the GP. Another idea is to split the training set into two disjoint sets, one which is actually used for training, and the other, the validation set, which is used to monitor performance. This approach is known as *cross-validation*. In the following, these two techniques for the selection of hyperparameters are presented.

3.2.1 Log Marginal Likelihood Approach

A very common method for the optimization of the hyperparameters is by means of the *negative log marginal likelihood function*, often simply named as (neg. log) likelihood function. It is *marginal* since it is obtained through marginalization over the function f_{GP} . The marginal likelihood is the likelihood that the output data $Y \in \mathbb{R}^{n_{\mathcal{D}}}$ fits to the input data X with the hyperparameters $\boldsymbol{\varphi}$. It is given by

$$\log p(Y|X, \boldsymbol{\varphi}) = -\frac{1}{2}Y^\top (K + \sigma_n^2 I_{n_{\mathcal{D}}})^{-1}Y - \frac{1}{2} \log |K + \sigma_n^2 I_{n_{\mathcal{D}}}| - \frac{n_{\mathcal{D}}}{2} \log 2\pi. \quad (50)$$

A detailed derivation can be found in [Ras06]. The three terms of the marginal likelihood in (50) have the following roles:

- $\frac{1}{2}Y^\top (K + \sigma_n^2 I_{n_{\mathcal{D}}})^{-1}Y$ is the only term that depends on the output data Y and represents the data-fit.
- $\frac{1}{2} \log |K + \sigma_n^2 I_{n_{\mathcal{D}}}|$ penalizes the complexity depending on the kernel function and the input data X .
- $\frac{n_{\mathcal{D}}}{2} \log 2\pi$ is a normalization constant.

Remark 5. For the sake of notational simplicity, we suppress the dependency on the hyperparameters of the kernel function k whenever possible.

The optimal hyperparameters $\boldsymbol{\varphi}^* \in \Phi$ and signal noise σ_n^* in the sense of the likelihood are obtained as the minimum of the negative log marginal likelihood function

$$\begin{bmatrix} \boldsymbol{\varphi}^* \\ \sigma_n^* \end{bmatrix} = \arg \min_{\boldsymbol{\varphi} \in \Phi, \sigma_n \in \mathbb{R}_{\geq 0}} \log p(Y|X, \boldsymbol{\varphi}). \quad (51)$$

Since an analytic solution of the derivation of (50) is impossible, a gradient based optimization algorithm is typically used to minimize the function. However, the negative log likelihood is non-convex in general such that there is no guarantee to find the optimum φ^*, σ_n^* . In fact, every local minimum corresponds to a particular interpretation of the data. In the following example, we visualize how the hyperparameters affect the regression.

Example 8. A GPR with the squared exponential kernel is trained on eight data points. The signal variance is fixed to $\varphi_1 = 2.13$. First, we visualize the influence of the lengthscale. For this purpose, the signal noise is fixed to $\sigma_n = 0.21$. Figure 7 shows the posterior mean of the regression and the neg. log likelihood function. On the left side are three posterior means for different lengthscales. A short lengthscale results in overfitting whereas a large lengthscale smooths out the training data (black crosses). The dotted red function represents the mean with optimized lengthscale by a descent gradient algorithm with respect to (51). The right plot shows the neg. log likelihood over the signal variance φ_1 and lengthscale φ_2 . The minimum is here at $\varphi^* = [2.13, 1.58]^\top$.

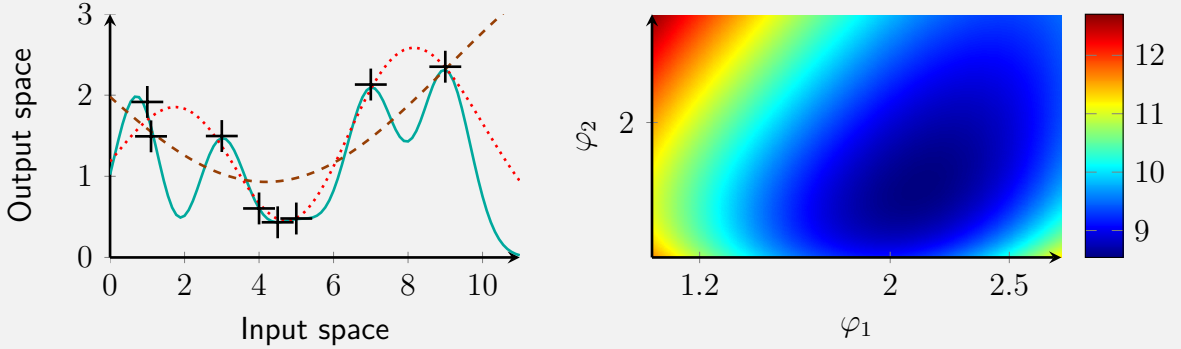


Figure 7: Left: Regression with different lengthscales: $\varphi_2 = 0.67$ (cyan, solid), $\varphi_2 = 7.39$ (brown, dashed), and $\varphi_2 = 1.58$ (red, dotted). Right: Neg. log likelihood function over signal variance φ_1 and lengthscale φ_2 .

Next, the meaning of different interpretations of the data is visualized by varying the signal noise σ_n and the lengthscale φ_2 . The right plot of Fig. 8 shows two minima of the negative log likelihood function. The lower left minimum at $\log(\sigma_n) = 0.73$ and $\log(\varphi_2) = -1.51$ interprets the data as slightly noisy which leads to the dotted red posterior mean in the left plot. In contrast, the upper right minimum at $\log(\sigma_n) = 5$ and $\log(\varphi_2) = -0.24$ interprets the data as very noisy without a trend, which manifests as the cyan posterior mean in the left plot. Depending on the initial value, a gradient based optimizer would terminate in one of these minima.

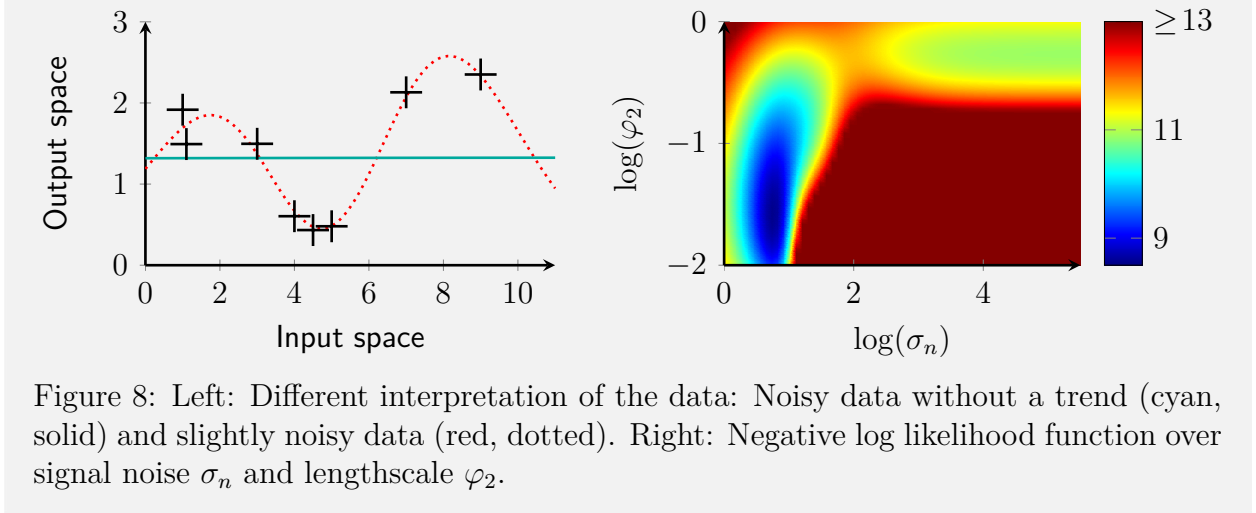


Figure 8: Left: Different interpretation of the data: Noisy data without a trend (cyan, solid) and slightly noisy data (red, dotted). Right: Negative log likelihood function over signal noise σ_n and lengthscale φ_2 .

3.2.2 Cross-validation Approach

This approach works with a separation of the data set $\mathcal{D}$ in two classes: one for training and one for validation. Cross-validation is almost always used in the k_{cv} -fold cross-validation setting: the k_{cv} -fold cross-validation data is split into k_{cv} disjoint, equally sized subsets; validation is done on a single subset and training is done using the union of the remaining $k_{cv} - 1$ subsets, the entire procedure is repeated k_{cv} times, each time with a different subset for validation. Here, without loss of generality, we present the leave-one-out cross-validation, which means $k_{cv} = n_{\mathcal{D}}$. The predictive log probability when leaving out a training point $\{\mathbf{x}_{\text{dat}}^{\{i\}}, \tilde{y}_{\text{dat}}^{\{i\}}\}$ is given by

$$\log p(y_{\text{dat}}^{\{i\}} | X, Y_{-i}, \boldsymbol{\varphi}) = -\frac{1}{2} \log(\text{var}_{-i}) - \frac{(\tilde{y}_{\text{dat}}^{\{i\}} - \mu_{-i})^2}{2 \text{var}_{-i}} - \frac{n_{\mathcal{D}}}{2} \log 2\pi, \quad (52)$$

where $\mu_{-i} = \mu(f_{\text{GP}}(\mathbf{x}_{\text{dat}}^{\{i\}}) | \mathbf{x}_{\text{dat}}^{\{i\}}, X_{:, -i}, Y_{-i})$ and $\text{var}_{-i} = \text{var}(f_{\text{GP}}(\mathbf{x}_{\text{dat}}^{\{i\}}) | \mathbf{x}_{\text{dat}}^{\{i\}}, X_{:, -i}, Y_{-i})$. The $-i$ index indicates X and Y without the element $\mathbf{x}_{\text{dat}}^{\{i\}}$ and $\tilde{y}_{\text{dat}}^{\{i\}}$, respectively. Thus, (52) is the probability for the output $y_{\text{dat}}^{\{i\}}$ at $\mathbf{x}_{\text{dat}}^{\{i\}}$ but without the training point $\{\mathbf{x}_{\text{dat}}^{\{i\}}, \tilde{y}_{\text{dat}}^{\{i\}}\}$. Accordingly, the leave-one-out log predictive probability $L_{\text{LOO}} \in \mathbb{R}$ is

$$L_{\text{LOO}} = \sum_{i=1}^{n_{\mathcal{D}}} \log p(y_{\text{dat}}^{\{i\}} | X, Y_{-i}, \boldsymbol{\varphi}). \quad (53)$$

In comparison to the log likelihood approach (51), the cross-validation is in general more computationally expensive but might find a better representation of the data set, see [GE79] for discussion and related approaches.

4 Gaussian Process Dynamical Models

So far, we consider GPR in non-dynamical settings where only an input-to-output mapping is considered. However, Gaussian process dynamical models (GPDMS) have recently become a versatile tool in system identification because of their beneficial properties such as the bias variance trade-off and the strong connection to Bayesian mathematics, see [FCR14].

In many works, where GPs are applied to dynamical model, only the mean function of the process is employed, e.g., in [WHB05] and [Cho+13]. This is mainly because GP models are often used to replace deterministic parametric models. However, GPDMS contain a much richer description of the underlying dynamics, but also the uncertainty about the model itself when the full probabilistic representation is considered. Therefore, one main aspect of GPDMS is to distinguish between recurrent structures and non-recurrent structures. A model is called recurrent if parts of the regression vector depend on the outputs of the model. Even though recurrent models become more complex in terms of their behavior, they allow to model sequences of data, see [Sjö+95]. If all states are fed back from the model itself, we get a simulation model, which is a special case of the recurrent structure. The advantage of such a model is its property to be independent from the real system. Thus, it is suitable for simulations, as it allows multi-step ahead predictions. In this report, we focus on two often-used recurrent structures: the Gaussian process state space model (GP-SSM) and the Gaussian process nonlinear error output (GP-NOE) model.

4.1 Gaussian Process State Space Models

Gaussian process state space models are structured as a discrete-time system. In this case, the states are the regressors, which is visualized in Fig. 9. This approach allows to be more efficient, since the regressors are less restricted in their internal structure as in input-output models. Thus, a very efficient model in terms of number of regressors might be possible. The mapping from the states to the output is often assumed to be known. The situation, where the output mapping describes a known sensor model, is such an example. It is mentioned in [Fri+13] that using too flexible models for both, the state mapping $\mathbf{f}$ and the output mapping, can result in problems of non-identifiability. Therefore, we focus on a known output mapping. The mathematical model of the GP-SSM is thus given by

$$\begin{aligned} \mathbf{x}_{t+1} = \mathbf{f}(\boldsymbol{\xi}_t) &= \begin{cases} f_1(\boldsymbol{\xi}_t) \sim \mathcal{GP}(m^1(\boldsymbol{\xi}_t), k^1(\boldsymbol{\xi}_t, \boldsymbol{\xi}'_t)) \\ \vdots \quad \quad \quad \vdots \\ f_{n_x}(\boldsymbol{\xi}_t) \sim \mathcal{GP}(m^{n_x}(\boldsymbol{\xi}_t), k^{n_x}(\boldsymbol{\xi}_t, \boldsymbol{\xi}'_t)). \end{cases} \\ \mathbf{y}_t &\sim p(\mathbf{y}_t | \mathbf{x}_t, \boldsymbol{\gamma}_y), \end{aligned} \tag{54}$$

where $\boldsymbol{\xi}_t \in \mathbb{R}^{n_\xi}$, $n_\xi = n_x + n_u$ is the concatenation of the state vector $\mathbf{x}_t \in \mathcal{X} \subseteq \mathbb{R}^{n_x}$ and the input $\mathbf{u}_t \in \mathcal{U} \subseteq \mathbb{R}^{n_u}$ such that $\boldsymbol{\xi}_t = [\mathbf{x}_t; \mathbf{u}_t]$. The mean function is given by continuous functions $m^1, \dots, m^{n_x}: \mathbb{R}^{n_\xi} \rightarrow \mathbb{R}$. The output mapping is parametrized by a known vector $\boldsymbol{\gamma}_y \in \mathbb{R}^{n_\gamma}$ with $n_\gamma \in \mathbb{N}$. The system identification task for the GP-SSM mainly focuses on $\mathbf{f}$ in particular. It can be described as finding the state-transition probability conditioned on the observed training data.

Remark 6. *The potentially unknown number of regressors can be determined using established nonlinear identification techniques as presented in [KL99], or exploiting embedded techniques such as automatic relevance determination [Koc16]. A mismatch leads to similar issues as in parametric system identification.*

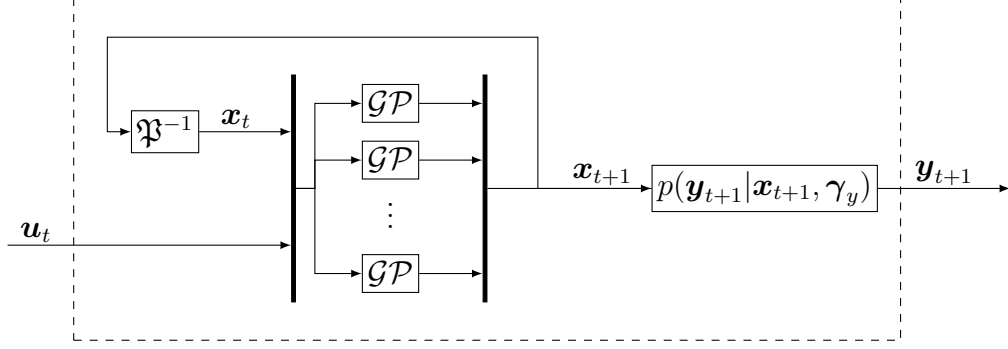


Figure 9: Structure of a GP-SSM with $\mathfrak{P}$ as backshift operator, such that $\mathfrak{P}^{-1}x_{t+1} = x_t$

4.2 Gaussian Process Nonlinear Output Error Models

The GP-NOE model uses the past $n_{\text{in}} \in \mathbb{N}_{>0}$ input values $\mathbf{u}_t \in \mathcal{U}$ and the past $n_{\text{out}} \in \mathbb{N}_{>0}$ output values $\mathbf{y}_t \in \mathbb{R}^{n_y}$ of the model as the regressors. Figure 10 shows the structure of GP-NOE, where the outputs are feedbacked. Analogously to the GP-SSM, the mathematical model of the GP-NOE is given by

$$\mathbf{y}_{t+1} = \mathbf{h}(\boldsymbol{\zeta}_t) = \begin{cases} h_1(\boldsymbol{\zeta}_t) \sim \mathcal{GP}(m^1(\boldsymbol{\zeta}_t), k^1(\boldsymbol{\zeta}_t, \boldsymbol{\zeta}'_t)) \\ \vdots \\ h_{n_y}(\boldsymbol{\zeta}_t) \sim \mathcal{GP}(m^{n_y}(\boldsymbol{\zeta}_t), k^{n_y}(\boldsymbol{\zeta}_t, \boldsymbol{\zeta}'_t)), \end{cases} \quad (55)$$

where $\boldsymbol{\zeta}_t \in \mathbb{R}^{n_\zeta}$, $n_\zeta = n_{\text{out}}n_y + n_{\text{in}}n_u$ is the concatenation of the past outputs $\mathbf{y}_t$ and inputs $\mathbf{u}_t$ such that $\boldsymbol{\zeta}_t = [\mathbf{y}_{t-n_{\text{out}}+1}; \dots; \mathbf{y}_t; \mathbf{u}_{t-n_{\text{in}}+1}; \dots; \mathbf{u}_t]$. The mean function is given by continuous functions $m^1, \dots, m^{n_y}: \mathbb{R}^{n_\zeta} \rightarrow \mathbb{R}$. In contrast to nonlinear autoregressive exogenous models, that focus on one-step ahead prediction, a NOE model is more suitable for simulations as it considers the multi-step ahead prediction [Nel13]. However, the drawback is a more complex training procedure that requires a nonlinear optimization scheme due to their recurrent structure [Koc16].

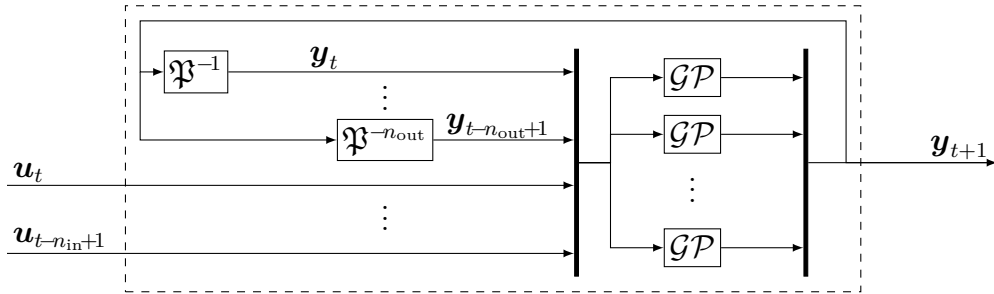


Figure 10: Structure of a GP-NOE model with $\mathfrak{P}$ as backshift operator ($\mathfrak{P}^{-1}\mathbf{y}_{t+1} = \mathbf{y}_t$)

Remark 7. It is always possible to convert an identified input-output model into a state-space model, see [PL70]. However, focusing on state-space models only would preclude the development of a large number of useful identification results.

Remark 8. Control relevant properties of GP-SSMs and GP-NOE models are discussed in [BH16b; BH16a; BH20].

5 Summary

In this article, we introduce the GP and its usage in GPR. Based on the property, that every finite subset of a GP follows a multi-variate Gaussian distribution, a closed-form equation can be derived to predict the mean and variance for a new test point. The GPR can intrinsically handle noisy output data if it is Gaussian distributed. As GPR is a data-driven method, only little prior knowledge is necessary for the regression. Further, the complexity of GP models scales with the number of training points. A degree of freedom in the modeling is the selection of the kernel function and its hyperparameters. We present an overview of common kernels and the necessary properties to be a valid kernel function. For the hyperparameter determination, two approaches based on numerical optimization are shown. The kernel of the GP is uniquely related to a RKHS, which determines the shape of the samples of the GP. Based on this, we compare different approaches for the quantification of the model error that quantifies the error between the GPR and the actual data-generating function. Finally, we introduce how GP models can be used as dynamical systems in GP-SSMs and GP-NOE models.

A Conditional Distribution

Let $\boldsymbol{\nu}_1 \in \mathbb{R}^{n_{\nu_1}}, \boldsymbol{\nu}_2 \in \mathbb{R}^{n_{\nu_2}}$ with $n_{\nu_1}, n_{\nu_2} \in \mathbb{N}$ be probability variables, which are multivariate Gaussian distribution

$$\begin{bmatrix} \boldsymbol{\nu}_1 \\ \boldsymbol{\nu}_2 \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12}^\top \\ \Sigma_{12} & \Sigma_{22} \end{bmatrix} \right) \quad (56)$$

with mean $\boldsymbol{\mu}_1 \in \mathbb{R}^{n_{\nu_1}}, \boldsymbol{\mu}_2 \in \mathbb{R}^{n_{\nu_2}}$ and variance $\Sigma_{11} \in \mathbb{R}^{n_{\nu_1} \times n_{\nu_1}}, \Sigma_{12} \in \mathbb{R}^{n_{\nu_2} \times n_{\nu_1}}, \Sigma_{22} \in \mathbb{R}^{n_{\nu_2} \times n_{\nu_2}}$. The task is to determine the conditional probability

$$p(\boldsymbol{\nu}_2 | \boldsymbol{\nu}_1) = \frac{p(\boldsymbol{\nu}_1, \boldsymbol{\nu}_2)}{p(\boldsymbol{\nu}_1)}. \quad (57)$$

The joined probability $p(\boldsymbol{\nu}_1, \boldsymbol{\nu}_2)$ is a multivariate Gaussian distribution with

$$p(\boldsymbol{\nu}_1, \boldsymbol{\nu}_2) = \frac{1}{(2\pi)^{(n_{\nu_1} + n_{\nu_2})/2} \det(\Sigma)^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right) \quad (58)$$

$$\boldsymbol{\mu} := \begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix}, \quad \Sigma := \begin{bmatrix} \Sigma_{11} & \Sigma_{12}^\top \\ \Sigma_{12} & \Sigma_{22} \end{bmatrix}, \quad (59)$$

where $\mathbf{x} = [\mathbf{x}_1; \mathbf{x}_2], \mathbf{x}_1 \in \mathbb{R}^{n_{\nu_1}}, \mathbf{x}_2 \in \mathbb{R}^{n_{\nu_2}}$. The marginal distribution of $\boldsymbol{\nu}_1$ is defined by the mean $\boldsymbol{\mu}_1$ and the variance Σ_{11} such that

$$p(\boldsymbol{\nu}_1) = \frac{1}{(2\pi)^{\frac{n_{\nu_1}}{2}} \det(\Sigma_{11})^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (\mathbf{x}_1 - \boldsymbol{\mu}_1)^\top \Sigma_{11}^{-1} (\mathbf{x}_1 - \boldsymbol{\mu}_1) \right). \quad (60)$$

The division of the joint distribution by the marginal distribution results again in a Gaussian distribution with

$$p(\boldsymbol{\nu}_2 | \boldsymbol{\nu}_1) = \underbrace{\frac{\det(\Sigma_{11})^{\frac{1}{2}}}{(2\pi)^{\frac{n_{\nu_2}}{2}} \det(\Sigma)^{\frac{1}{2}}}}_{*} \exp \left(-\frac{1}{2} \underbrace{\left[(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) - (\mathbf{x}_1 - \boldsymbol{\mu}_1)^\top \Sigma_{11}^{-1} (\mathbf{x}_1 - \boldsymbol{\mu}_1) \right]}_{**} \right), \quad (61)$$

where the first part $*$ can be rewritten as

$$* = \frac{1}{(2\pi)^{\frac{n_{\nu_2}}{2}}} \left(\frac{\det(\Sigma_{11})}{\det(\Sigma_{11}) \det(\Sigma_{22} - \Sigma_{12} \Sigma_{11}^{-1} \Sigma_{12}^\top)} \right)^{\frac{1}{2}} = \frac{1}{(2\pi)^{\frac{n_{\nu_2}}{2}} \det(\Sigma_{22} - \Sigma_{12} \Sigma_{11}^{-1} \Sigma_{12}^\top)^{\frac{1}{2}}}. \quad (62)$$

Thus, the covariance matrix $\Sigma_{22|1}$ of the conditional distribution $p(\boldsymbol{\nu}_2 | \boldsymbol{\nu}_1)$ is given by

$$\Sigma_{22|1} = \Sigma_{22} - \Sigma_{12} \Sigma_{11}^{-1} \Sigma_{12}^\top. \quad (63)$$

For the simplification of the second part $**$ of (61), we exploit the special block structure of Σ , such that its inverse is given by

$$\Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}, \quad \Sigma^{-1} = \begin{bmatrix} \Sigma'_{11} & \Sigma'_{12} \\ \Sigma'_{21} & \Sigma'_{22} \end{bmatrix} \quad (64)$$

$$\begin{aligned} \Sigma'_{11} &= \Sigma_{11}^{-1} + \Sigma_{11}^{-1} \Sigma_{12} N \Sigma_{21} \Sigma_{11}^{-1} \\ \Sigma'_{12} &= -\Sigma_{11}^{-1} \Sigma_{12} N \\ \Sigma'_{21} &= -N \Sigma_{21} \Sigma_{11}^{-1} \\ \Sigma'_{22} &= N \end{aligned} \quad (65)$$

with $N = (\Sigma_{22} - \Sigma_{21}\Sigma_{11}^{-1}\Sigma_{12})^{-1}$. Thus, we compute $**$ as

$$** = \begin{bmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{bmatrix}^\top \begin{bmatrix} \Sigma_{11} & \Sigma_{12}^\top \\ \Sigma_{12} & \Sigma_{22} \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{x}_1 - \boldsymbol{\mu}_1 \\ \mathbf{x}_2 - \boldsymbol{\mu}_2 \end{bmatrix} - (\mathbf{x}_1 - \boldsymbol{\mu}_1)^\top \Sigma_{11}^{-1} (\mathbf{x}_1 - \boldsymbol{\mu}_1) \quad (66)$$

$$= (\mathbf{x}_2 - \boldsymbol{\mu}_2)^\top \Sigma_{22|1}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2) + 2(\mathbf{x}_2 - \boldsymbol{\mu}_2)^\top \left(-\Sigma_{11}^{-1} \Sigma_{12}^\top \Sigma_{22|1}^{-1} \right) (\mathbf{x}_1 - \boldsymbol{\mu}_1) \\ + (\mathbf{x}_1 - \boldsymbol{\mu}_1)^\top \left(\Sigma_{11}^{-1} + \Sigma_{11}^{-1} \Sigma_{12}^\top \Sigma_{22|1}^{-1} \Sigma_{12} \Sigma_{11}^{-1} \right) (\mathbf{x}_1 - \boldsymbol{\mu}_1) - (\mathbf{x}_1 - \boldsymbol{\mu}_1)^\top \Sigma_{11}^{-1} (\mathbf{x}_1 - \boldsymbol{\mu}_1) \quad (67)$$

$$= \left(\mathbf{x}_2 - \underbrace{\boldsymbol{\mu}_2 + \Sigma_{12} \Sigma_{11}^{-1} (\mathbf{x}_1 - \boldsymbol{\mu}_1)}_{\boldsymbol{\mu}_{2|1}} \right)^\top \Sigma_{22|1}^{-1} \left(\mathbf{x}_2 - \underbrace{\boldsymbol{\mu}_2 + \Sigma_{12} \Sigma_{11}^{-1} (\mathbf{x}_1 - \boldsymbol{\mu}_1)}_{\boldsymbol{\mu}_{2|1}} \right) \quad (68)$$

Finally, the conditional probability is given with the conditional mean $\boldsymbol{\mu}_{2|1} \in \mathbb{R}^{n_{\nu_2}}$ and the conditional covariance matrix $\Sigma_{22|1} \in \mathbb{R}^{n_{\nu_2} \times n_{\nu_2}}$ by

$$p(\boldsymbol{\nu}_2 | \boldsymbol{\nu}_1) = \frac{1}{(2\pi)^{\frac{n_{\nu_2}}{2}} \det(\Sigma_{22|1})^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (\mathbf{x}_2 - \boldsymbol{\mu}_{2|1})^\top \Sigma_{22|1}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_{2|1}) \right) \quad (69)$$

$$\begin{aligned} \boldsymbol{\mu}_{2|1} &= \boldsymbol{\mu}_2 + \Sigma_{12} \Sigma_{11}^{-1} (\mathbf{x}_1 - \boldsymbol{\mu}_1) \\ \Sigma_{22|1} &= \Sigma_{22} - \Sigma_{12} \Sigma_{11}^{-1} \Sigma_{12}^\top. \end{aligned} \quad (70)$$

References

- [ÁRL12] Mauricio A. Álvarez, Lorenzo Rosasco, and Neil D. Lawrence. “Kernels for Vector-Valued Functions: A Review”. In: *Foundations and Trends in Machine Learning* 4.3 (2012), pp. 195–266. DOI: 10.1561/22000000036.
- [Aro50] Nachman Aronszajn. “Theory of reproducing kernels”. In: *Transactions of the American mathematical society* 68.3 (1950), pp. 337–404. DOI: 10.2307/1990404.
- [Ber+16] Felix Berkenkamp, Riccardo Moriconi, Angela P. Schoellig, and Andreas Krause. “Safe Learning of Regions of Attraction for Uncertain, Nonlinear Systems with Gaussian Processes”. In: *2016 IEEE 55th Conference on Decision and Control (CDC)*. 2016, pp. 4661–4666.
- [Ber+17] Felix Berkenkamp, Matteo Turchetta, Angela P. Schoellig, and Andreas Krause. “Safe Model-based Reinforcement Learning with Stability Guarantees”. In: *Advances in Neural Information Processing Systems*. 2017, pp. 908–918.
- [BH16a] Thomas Beckers and Sandra Hirche. “Equilibrium distributions and stability analysis of Gaussian Process State Space Models”. In: *2016 IEEE 55th Conference on Decision and Control (CDC)*. 2016, pp. 6355–6361. DOI: 10.1109/CDC.2016.7799247.
- [BH16b] Thomas Beckers and Sandra Hirche. “Stability of Gaussian Process State Space Models”. In: *2016 European Control Conference (ECC)*. 2016, pp. 2275–2281. DOI: 10.1109/ECC.2016.7810630.
- [BH20] Thomas Beckers and Sandra Hirche. “Prediction with Gaussian Process Dynamical Models”. In: *Transaction on Automatic Control* (2020). Submitted.
- [Bis06] Christopher Bishop. *Pattern recognition and machine learning*. Springer-Verlag New York, 2006. ISBN: 9780387310732.
- [BKH19] Thomas Beckers, Dana Kulić, and Sandra Hirche. “Stable Gaussian Process based Tracking Control of Euler-Lagrange Systems”. In: *Automatica* 103 (2019), pp. 390–397. DOI: 10.1016/j.automatica.2019.01.023.
- [BLG16] Yusuf Bhujwala, Vincent Laurain, and Marion Gilson. “The impact of smoothness on model class selection in nonlinear system identification: An application of derivatives in the RKHS”. In: *2016 American Control Conference (ACC)*. 2016, pp. 1808–1813. DOI: 10.1109/ACC.2016.7525181.
- [BUH18] Thomas Beckers, Jonas Umlauft, and Sandra Hirche. “Mean Square Prediction Error of Misspecified Gaussian Process Models”. In: *2018 IEEE Conference on Decision and Control (CDC)*. 2018, pp. 1162–1167. DOI: 10.1109/CDC.2018.8619163.
- [Cho+13] Girish Chowdhary, Hassan Kingravi, Jonathan How, and Patricio A. Vela. “Bayesian nonparametric adaptive control of time-varying systems using Gaussian processes”. In: *2013 American Control Conference*. 2013, pp. 2655–2661. DOI: 10.1109/ACC.2013.6580235.
- [DM+05] Lokenath Debnath, Piotr Mikusinski, et al. *Introduction to Hilbert spaces with applications*. Academic press, 2005.

- [FCR14] Roger Frigola, Yutian Chen, and Carl E. Rasmussen. *Variational Gaussian Process State-Space Models*. 2014. arXiv: 1406.4905 [cs.LG].
- [Fri+13] Roger Frigola, Fredrik Lindsten, Thomas B. Schön, and Carl E. Rasmussen. “Bayesian inference and learning in Gaussian process state-space models with particle MCMC”. In: *Advances in Neural Information Processing Systems*. 2013, pp. 3156–3164.
- [GE79] Seymour Geisser and William F. Eddy. “A Predictive Approach to Model Selection”. In: *Journal of the American Statistical Association* 74.365 (1979), pp. 153–160. DOI: 10.1080/01621459.1979.10481632.
- [KL99] Robert Keviczky and Haber Laszlo. *Nonlinear system identification: input-output modeling approach*. Springer Netherlands, 1999. ISBN: 9780792358589.
- [Koc16] Juš Kocijan. *Modelling and Control of Dynamic Systems Using Gaussian Process Models*. Springer International Publishing, 2016. DOI: 10.1007/978-3-319-21021-6.
- [Mac97] David J. MacKay. *Gaussian Processes - A Replacement for Supervised Neural Networks?* 1997.
- [MXZ06] Charles A. Micchelli, Yuesheng Xu, and Haizhang Zhang. “Universal kernels”. In: *Journal of Machine Learning Research* 7 (2006), pp. 2651–2667.
- [Nel13] Oliver Nelles. *Nonlinear system identification: from classical approaches to neural networks and fuzzy models*. Springer-Verlag Berlin Heidelberg, 2013. DOI: 10.1007/978-3-662-04323-3.
- [PL70] M. Q. Phan and R. W. Longman. “Relationship between state-space and input-output models via observer Markov parameters”. In: *WIT Transactions on The Built Environment* 22 (1970). DOI: 10.2495/DCSS960121.
- [Rad96] Neal M. Radford. *Bayesian learning for neural networks*. Springer-Verlag New York, 1996. DOI: 10.1007/978-1-4612-0745-0.
- [Ras06] Carl E. Rasmussen. *Gaussian processes for machine learning*. The MIT Press, 2006. ISBN: 9780262182539.
- [SC04] John Shawe-Taylor and Nello Cristianini. *Kernel methods for pattern analysis*. Cambridge university press, 2004. DOI: 10.1017/CB09780511809682.
- [SC08] Ingo Steinwart and Andreas Christmann. *Support vector machines*. Springer Science & Business Media, 2008. DOI: 10.1007/978-0-387-77242-4.
- [SHS06] Ingo Steinwart, Don Hush, and Clint Scovel. “An explicit description of the reproducing kernel Hilbert spaces of Gaussian RBF kernels”. In: *IEEE Transactions on Information Theory* 52.10 (2006), pp. 4635–4643. DOI: 10.1109/TIT.2006.881713.
- [Sjö+95] Jonas Sjöberg, Qinghua Zhang, Lennart Ljung, Albert Benveniste, Bernard Delyon, Pierre-Yves Glorennec, Håkan Hjalmarsson, and Anatoli Juditsky. “Non-linear black-box modeling in system identification: a unified overview”. In: *Automatica* 31.12 (1995), pp. 1691–1724. DOI: 10.1016/0005-1098(95)00120-8.

- [Sri+12] Niranjana Srinivas, Andreas Krause, Sham M. Kakade, and Matthias W. Seeger. “Information-theoretic regret bounds for Gaussian process optimization in the bandit setting”. In: *IEEE Transactions on Information Theory* 58.5 (2012), pp. 3250–3265. DOI: 10.1109/TIT.2011.2182033.
- [UBH18] Jonas Umlauft, Thomas Beckers, and Sandra Hirche. “A Scenario-based Optimal Control Approach for Gaussian Process State Space Models”. In: *2018 European Control Conference (ECC)*. 2018, pp. 1386–1392. DOI: 10.23919/ECC.2018.8550458.
- [Wah90] Grace Wahba. *Spline models for observational data*. SIAM, 1990. DOI: 10.1137/1.9781611970128.
- [WHB05] Jack Wang, Aaron Hertzmann, and David M. Blei. “Gaussian process dynamical models”. In: *Proceedings of the 18th International Conference on Neural Information Processing System*. 2005, pp. 1441–1448. DOI: 10.5555/2976248.2976429.
- [WR96] Christopher K. Williams and Carl E. Rasmussen. “Gaussian processes for regression”. In: *Advances in neural information processing systems*. 1996, pp. 514–520.